Ricerca Operativa

Diego Stefanini — prof. Stefano Novellani, a.a. 2026-27

Indice

1 Tecniche di modellazione 3
1.1 Il problema della fonderia (mix di produzione) 3
1.2 Variabili binarie: zaino e relazioni logiche 5
1.3 Scegliere lati di un grafo: albero di copertura e commesso viaggiatore 7
1.4 Assegnamento, semi-assegnamento e colorazione di grafi 10
1.5 Selezione di sottoinsiemi: partizione, copertura, riempimento 12
1.6 Problemi di flusso su rete 18
1.7 Variabili discrete, semicontinue e funzioni a tratti 20
1.8 Linearizzazioni: min-max, valore assoluto, vincoli disgiuntivi 24
2 Grafi e reti di flusso 30
2.1 Reti di flusso e trasformazioni equivalenti 30
2.2 Cammini minimi: il problema e le condizioni di Bellman 32
2.3 Algoritmi per l'albero dei cammini minimi: Bellman-Ford e Dijkstra 35
2.4 Cammini minimi su grafi aciclici 39
2.5 Albero di copertura di costo minimo 40
Riepilogo 44

1 Tecniche di modellazione

Cosa devi saper fare

- tradurre un testo in un modello: variabili con il loro dominio, funzione obiettivo, vincoli;
- scrivere con variabili binarie le relazioni logiche (almeno uno, implicazione, and, or, xor) e controllarle con la tabella di verità;
- legare due famiglie di variabili (frequenze e antenne, bonus e piatti, flusso e condotta);
- scrivere i vincoli di conservazione del flusso a partire dai deficit;
- modellare variabili discrete, semicontinue, costi di setup e funzioni a tratti;
- rendere lineari un min-max, un valore assoluto, un «oppure» fra vincoli, un prodotto di probabilità.

Un modello ha sempre tre pezzi: **variabili** (le decisioni), **funzione obiettivo** (cosa minimizzo o massimizzo), **vincoli** (cosa devo rispettare).

Da sapere — equazione lineare in più variabili (aggiunto, non spiegato in aula)
Un'espressione è **lineare** se ogni variabile compare **da sola, alla prima potenza, moltiplicata solo per un numero**, e i termini si sommano:

$$a_1x_1 + a_2x_2 + \ldots + a_nx_n = b$$

con $a_1, \ldots, a_n, b$ numeri fissi. Vale lo stesso per $\leq$ e $\geq$ (disequazioni lineari).

$4x_1 + x_2 + 0.6x_3 \geq 3250$	✓ lineare
$x_1 + 3 \geq 2x_2 - 7$	✓ lineare (sposto tutto a sinistra)
$x_1 \cdot x_2 \leq 5$	✗ prodotto di due variabili
$x_1^2 + x_2 \leq 5$	✗ potenza
$\frac{x_1}{x_1+x_2} \geq 0.2$	✗ divisione per variabili (ma si può rendere lineare, vedi sotto)

Perché conta: se funzione obiettivo e vincoli sono lineari, il problema è di **Programmazione Lineare** (PL) e ci sono algoritmi efficienti per risolverlo.

Cosa non si può scrivere in un modello (errori che all'esame escono sempre):

- solo $\leq$, $\geq$ e $=$: niente disuguaglianze strette ($<$, $>$) e niente $\neq$ («$x \neq y$» non si può scrivere);
- niente prodotti o divisioni fra variabili, niente valore assoluto $|x|$, niente min o max di variabili dentro i vincoli: solo espressioni lineari.

«s.t.» sotto la funzione obiettivo sta per *subject to*, «soggetto a»: da lì in poi ci sono i vincoli. Ricordarsi sempre di scrivere anche il **dominio** delle variabili, cioè i valori che possono assumere ($x \geq 0$, $x \in \{0, 1\}$...).

Lo stesso problema si può scrivere con **modelli diversi**: nel corso l'efficienza non conta, basta che il modello funzioni (anche con qualche vincolo in più).

1.1 Il problema della fonderia (mix di produzione)

x_1 kg di **materiale 1**	4% Si · 0,45% Mn · 0,025 €/kg
x_2 kg di **materiale 2**	1% Si · 0,50% Mn · 0,030 €/kg
x_3 kg di **materiale 3**	0,6% Si · 0,40% Mn · 0,018 €/kg
x_4 kg di **manganese puro**	100% Mn · 10 €/kg

→ **1000 pezzi da 1 kg**
silicio fra 3,25% e 5,5%
manganese almeno 0,45%

Variabili: x_1, x_2, x_3 = kg di materiale ferroso 1, 2, 3 nel mix; x_4 = kg di manganese puro.

Da percentuali a vincoli: il silicio nel mix, in kg, è $0.04x_1 + 0.01x_2 + 0.006x_3$. Deve essere almeno il 3,25% di 1000 kg, cioè 32,5 kg. Moltiplico tutto per 1000 per togliere le virgole:

$$\begin{aligned} 0.04x_1 + 0.01x_2 + 0.006x_3 &\geq 32.5 \\ 40x_1 + 10x_2 + 6x_3 &\geq 32500 \quad (\times 1000) \end{aligned}$$

Stesso ragionamento per il manganese: $0.0045x_1 + 0.005x_2 + 0.004x_3 + x_4 \geq 4.5$, moltiplicato per 10 000.

$$\begin{aligned} \min z = {} & 0.025x_1 + 0.030x_2 + 0.018x_3 + 10x_4 \\ & x_1 + x_2 + x_3 + x_4 = 1000 && \leftarrow \text{minimizzo il costo} \\ & 40x_1 + 10x_2 + 6x_3 \geq 32500 && \leftarrow \text{totale da produrre} \\ & 40x_1 + 10x_2 + 6x_3 \leq 55000 && \leftarrow \text{silicio minimo} \\ & 45x_1 + 50x_2 + 40x_3 + 10000x_4 \geq 45000 && \leftarrow \text{silicio massimo} \\ & x_1, x_2, x_3, x_4 \geq 0 && \leftarrow \text{manganese minimo} \\ & && \leftarrow \text{non negatività} \end{aligned}$$

Il modello si può scrivere in **forma generale**: al posto dei numeri metto dei **parametri**: $n = 4$ materiali, $K = 1000$ kg, c_i costo al kg, s_i e m_i percentuali di silicio e manganese del materiale i, $\underline{s}$ e $\overline{s}$ silicio minimo e massimo, $\underline{m}$ manganese minimo.

$$\min z = \sum_{i=1}^{n} c_i x_i \quad \text{s.t.} \quad \sum_{i=1}^{n} x_i = K, \quad \underline{s}K \leq \sum_{i=1}^{n-1} s_i x_i \leq \overline{s}K, \quad \sum_{i=1}^{n} m_i x_i \geq \underline{m}K, \quad x_i \geq 0$$

> La somma del silicio arriva a $n-1$: il manganese puro (materiale 4) non contiene silicio.
>
> Un'**istanza** è una configurazione concreta del problema: il modello generale con dei valori al posto dei parametri. Quello con i numeri sopra è un'istanza.

Un vincolo che capita spesso è quello **in percentuale**:

«Il materiale 1 deve essere almeno il 20% dei materiali ferrosi usati.»

$$\frac{x_1}{x_1 + x_2 + x_3} \geq 0.2$$

non lineare: divido per variabili

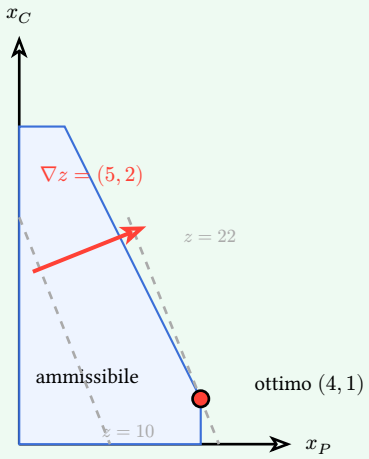
moltiplico per il denominatore $\longrightarrow$

$$x_1 \geq 0.2(x_1 + x_2 + x_3)$$

lineare

> Si può moltiplicare senza girare il $\geq$ perché il denominatore è una somma di quantità ≥ 0. Lo stesso trucco serve quando chiedono una **media pesata**.

> **Da sapere — gradiente** (aggiunto, non spiegato in aula)
> Per una funzione lineare $z = c_1x_1 + c_2x_2$ il **gradiente** è il vettore dei coefficienti, $\nabla z = (c_1, c_2)$. Indica la direzione in cui z **cresce più in fretta**.



Esempio: $\max z = 5x_P + 2x_C$ con $x_P \leq 4$, $x_C \leq 7$, $2x_P + x_C \leq 9$.

- Le rette tratteggiate sono **linee di livello**: punti con lo stesso z. Sono perpendicolari al gradiente.
- Per un **max** sposto la retta nella direzione del gradiente finché tocca ancora la regione: l'ultimo punto toccato è l'ottimo, $(4, 1)$ con $z = 22$.
- Per un **min** vado nella direzione opposta, $-\nabla z$.

1.2 Variabili binarie: zaino e relazioni logiche

Un investitore ha un capitale B e n progetti. Il progetto i costa c_i e rende w_i. Quali progetti finanzio per rendere il più possibile senza sforare il budget?

budget B

$x_1 = 1$	$x_3 = 1$	$x_4 = 1$	avanzo

restano fuori: $x_2 = 0$, $x_5 = 0$

$$x_i = \begin{cases} 1 \text{ se finanzio il progetto } i \\ 0 \text{ altrimenti} \end{cases}$$

$$\max z = \sum_{i=1}^{n} w_i x_i$$

$$\text{s.t.} \sum_{i=1}^{n} c_i x_i \leq B$$

$$x_i \in \{0, 1\} \quad \forall i = 1, \ldots, n$$

Le variabili **binarie** (o booleane) servono per le decisioni sì/no.

Il trucco: moltiplicando per x_i, nella somma entrano **solo i progetti scelti** ($x_i = 1$); gli altri valgono 0.

$\sum c_i x_i \leq B$ è il **vincolo di budget**.

Con le variabili binarie si scrivono anche le **relazioni logiche** fra le decisioni: «se finanzio questo devo finanziare quello», «al massimo uno fra questi». Ogni relazione diventa un vincolo lineare. Tutte le variabili qui sono binarie: $x_A = 1$ se finanzio il progetto A. Nelle tabelle, ✓ = combinazione permessa dal vincolo.

Sono esempi separati: non devono valere tutti insieme.

Il caso più semplice: **esattamente, almeno o al massimo uno** fra A e B.

x_A	x_B	$x_A + x_B = 1$	$x_A + x_B \geq 1$	$x_A + x_B \leq 1$
0	0	✗	✗	✓
0	1	✓	✓	✓
1	0	✓	✓	✓
1	1	✗	✓	✗

1. **esattamente uno** fra A e B: $x_A + x_B = 1$
2. **almeno uno**: $x_A + x_B \geq 1$
3. **al massimo uno**: $x_A + x_B \leq 1$

Funziona con più variabili e un termine noto diverso: «non più di due fra A, B, C» è $x_A + x_B + x_C \leq 2$.

Negazione: B = non A si scrive $x_B = 1 - x_A$ (è il caso 1).

Implicazione.

x_A	x_C	$x_A \leq x_C$
0	0	✓
0	1	✓
1	0	✗
1	1	✓

4. «Se investo in A devo investire in C; se non investo in A, C è libero»: $x_A \leq x_C$.

Se $x_A = 1$ il vincolo forza $x_C = 1$; se $x_A = 0$ diventa $0 \leq x_C$, sempre vero.

Il «se e solo se» (relazione biunivoca) è l'uguaglianza: $x_A = x_C$.

And: si vuole $C = A \wedge B$.

x_A	x_B	x_C	5	6	5 + 6
0	0	0	✓	✓	✓
0	0	1	✓	✗	✗
0	1	0	✓	✓	✓
0	1	1	✓	✗	✗
1	0	0	✓	✓	✓
1	0	1	✓	✗	✗
1	1	0	✗	✓	✗
1	1	1	✓	✓	✓

5. «Se finanzio **sia A che B** devo finanziare C; altrimenti C è libero»: $x_C \geq x_A + x_B - 1$
   (a destra c'è 1 solo se $x_A = x_B = 1$, altrimenti ≤ 0)
6. «Posso finanziare C **solo se** finanzio A e B contemporaneamente»: $x_C \leq x_A$ e $x_C \leq x_B$

Insieme danno $C = A \wedge B$: la colonna 5 + 6 è ✓ solo dove $x_C = x_A \cdot x_B$.

Con tre progetti: «se finanzio A, B e C devo finanziare D» è $x_D \geq x_A + x_B + x_C - 2$. In generale con k progetti il -1 diventa $-(k-1)$.

> I due vincoli del 6 si possono sommare in uno solo: $2x_C \leq x_A + x_B$, cioè $x_C \leq \frac{x_A + x_B}{2}$. Se almeno uno fra A e B vale 0, a destra c'è al massimo $\frac{1}{2}$, e una variabile binaria $\leq \frac{1}{2}$ può solo valere 0. Stessa cosa per il 7 più avanti: $2x_C \geq x_A + x_B$. Per l'esame le due scritture valgono uguale.

Or: si vuole $C = A \vee B$.

x_A	x_B	x_C	7	8	7 + 8
0	0	0	✓	✓	✓
0	0	1	✓	✗	✗
0	1	0	✗	✓	✗
0	1	1	✓	✓	✓
1	0	0	✗	✓	✗
1	0	1	✓	✓	✓
1	1	0	✗	✓	✗
1	1	1	✓	✓	✓

7. «**Devo** finanziare C se finanzio almeno uno fra A e B»: $x_C \geq x_A$ e $x_C \geq x_B$
8. «**Posso** finanziare C solo se finanzio almeno uno fra A e B»: $x_C \leq x_A + x_B$

Insieme danno $C = A \vee B$.

Or esclusivo: si vuole C finanziato quando c'è esattamente uno fra A e B.

x_A	x_B	x_C	9	10	9 + 10
0	0	0	✓	✓	✓
0	0	1	✓	✗	✗
0	1	0	✗	✓	✗
0	1	1	✓	✓	✓
1	0	0	✗	✓	✗
1	0	1	✓	✓	✓
1	1	0	✓	✓	✓
1	1	1	✓	✗	✗

9. «**Devo** finanziare C se finanzio esattamente uno fra A e B»: $x_C \geq x_A - x_B$ e $x_C \geq x_B - x_A$
10. «**Posso** finanziare C solo se finanzio esattamente uno fra A e B»: $x_C \leq 2 - x_A - x_B$ e $x_C \leq x_A + x_B$

Insieme danno $C = A \oplus B$.

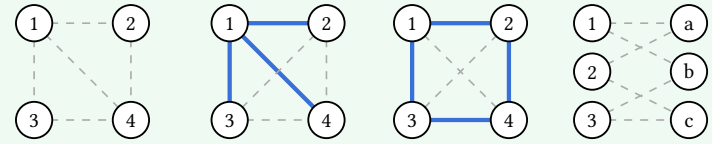
> Trappola sul 9: $x_C \geq x_A + x_B$ è sbagliato. Con A e B entrambi finanziati chiede $x_C \geq 2$, impossibile per una binaria: il problema diventa **inammissibile** (nessuna soluzione rispetta tutti i vincoli). In quel caso invece C deve restare libero.

> Schema che si ripete: il «**devo**» (se … allora C) è un vincolo $x_C \geq \ldots$ che spinge C a 1; il «**posso solo se**» è un vincolo $x_C \leq \ldots$ che tiene C a 0.

1.3 Scegliere lati di un grafo: albero di copertura e commesso viaggiatore

Nello zaino le variabili binarie sceglievano un **sottoinsieme** dei progetti. Qui scelgono un sottoinsieme dei **lati di un grafo**, e i vincoli gli impongono una forma: prima un albero, poi un ciclo.

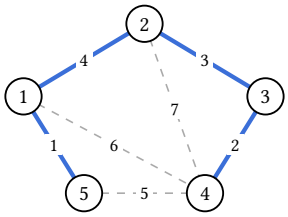
> **Da sapere — grafi** (aggiunto, non spiegato in aula)
> Un **grafo non orientato** $G = (V, E)$ è un insieme di **vertici** V (i pallini) e di **lati** E (le linee). Un lato unisce due vertici e si scrive $\{i, j\}$: le graffe dicono che l'ordine non conta, $\{i, j\}$ e $\{j, i\}$ sono lo stesso lato. Con $n = |V|$ si indica il numero di vertici ($|\cdot|$ = numero di elementi di un insieme). Nei grafi **orientati** le linee hanno un verso e si chiamano **archi**, (i, j); qui sono non orientati e si chiamano **lati** (se il prof dice «arco», qui leggi «lato»).



un grafo connesso | grafo completo, in blu un albero | in blu un ciclo hamiltoniano | grafo bipartito

- **connesso**: da ogni vertice si arriva a ogni altro camminando sui lati.
- **ciclo**: un giro di lati che torna al vertice di partenza. Un ciclo **hamiltoniano** passa per **tutti** i vertici, **una sola volta** ciascuno.
- **albero**: grafo connesso e **senza cicli**. Un albero su n vertici ha sempre $n-1$ lati. Un **albero di copertura** di G è un albero fatto con lati di G che tocca tutti i vertici.
- **completo**: c'è un lato fra ogni coppia di vertici.
- **bipartito**: i vertici si dividono in due gruppi e ogni lato va da un gruppo all'altro, mai dentro lo stesso gruppo.
- **taglio**: preso un sottoinsieme S di vertici, $V \setminus S$ sono i vertici **fuori** da S; i lati del taglio sono quelli con un estremo in S e l'altro in $V \setminus S$, cioè quelli che «escono» da S.

Albero di copertura di costo minimo (Minimum Spanning Tree, MST). Il grafo $G = (V, E)$ è connesso e non orientato: i vertici $V = \{1, \dots, n\}$ sono città, i lati sono i collegamenti che si possono costruire, e il lato $\{i, j\}$ costa $c_{ij} > 0$. Si vuole che ogni città raggiunga ogni altra attraverso la rete, spendendo il meno possibile.



Variabili: una per lato,

$$x_{ij} = \begin{cases} 1 \text{ se costruisco il collegamento } \{i,j\} \in E \\ 0 \text{ altrimenti} \end{cases}$$

Si potrebbe usare una variabile per ogni albero possibile, ma gli alberi sono un numero esponenziale: meglio costruire la soluzione **a pezzettini**, un lato alla volta. Nel disegno i lati in blu hanno $x_{ij} = 1$: costo $1 + 4 + 3 + 2 = 10$, il minimo per questo grafo.

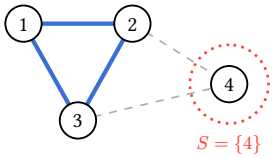
Funzione obiettivo: come nello zaino, nella somma entrano solo i lati scelti,

$$\min z = \sum_{\{i,j\} \in E} c_{ij} x_{ij}$$

Vincoli: come si scrive «la rete è connessa»? Qualunque gruppo di città S prenda, almeno un lato scelto deve **uscire** da S, sennò le città di S restano isolate. Vale per ogni S non vuoto e diverso da tutto V (con $S = V$ non c'è un «fuori»): si scrive $S \subset V$ (sottoinsieme **proprio**) e $S \neq \varnothing$:

$$\sum_{i \in S, j \in V \setminus S} x_{ij} \geq 1 \quad \forall S \subset V, S \neq \varnothing \qquad x_{ij} \in \{0, 1\} \quad \forall \{i, j\} \in E$$

La somma conta i lati scelti che attraversano il taglio: deve essere almeno 1.



Esempio: scelgo i lati del triangolo 1-2-3. Il vertice 4 resta fuori dalla rete. Con $S = \{4\}$ i lati che escono da S sono $\{2, 4\}$ e $\{3, 4\}$, entrambi con $x = 0$: la somma fa $0 < 1$, il vincolo è violato e la soluzione è esclusa.

Il **numero di vincoli è esponenziale**: uno per ogni sottoinsieme S. I sottoinsiemi di n vertici sono 2^n; tolti $\varnothing$ e V restano $2^n - 2$ vincoli. Con 30 città sono già più di un miliardo.

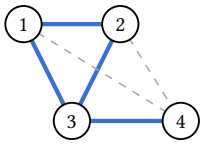
Il modello chiede solo che la rete sia **connessa**, non che sia un albero. Ma all'ottimo lo è per forza: se i lati scelti formassero un ciclo, togliendone uno la rete resterebbe connessa e costerebbe meno, perché ogni $c_{ij} > 0$.

Un'altra idea: un albero su n vertici ha sempre $n - 1$ lati, quindi si potrebbe scrivere $\sum_{\{i,j\}\in E} x_{ij} = n - 1$. **Da solo non basta**: $n - 1$ lati possono chiudere un ciclo e lasciare fuori un vertice (è il triangolo di prima: 3 lati, $n - 1 = 3$, ma il 4 è isolato). **Insieme ai vincoli di taglio è ridondante**: con costi positivi l'ottimo è già un albero. Servirebbe solo con costi negativi, perché allora al modello converrebbe prendere lati in più.

Problema del commesso viaggiatore (Travelling Salesman Problem, TSP). Il grafo $G = (V, E)$ è **completo** e non orientato: $V = \{1, \dots, n\}$ sono città, $E = \{\{i, j\} : i, j \in V, i \neq j\}$ contiene tutti i collegamenti possibili, ognuno con costo $c_{ij} \geq 0$. Si cerca un **ciclo hamiltoniano di costo minimo**: parto da una città, visito ciascuna delle altre **una e una sola volta**, torno alla città di partenza, e il giro costa il meno possibile.

Variabili e **funzione obiettivo** sono le stesse dell'albero: $x_{ij} = 1$ se uso il collegamento $\{i, j\}$, e $\min z = \sum_{\{i,j\}\in E} c_{ij} x_{ij}$. Cambiano i vincoli.

Senza vincoli il min non sceglierebbe nessun lato (costo 0). Prima idea: in un ciclo che passa per tutte le città ci sono tanti lati quanti vertici, quindi $\sum_{\{i,j\}\in E} x_{ij} = n$. Non basta:

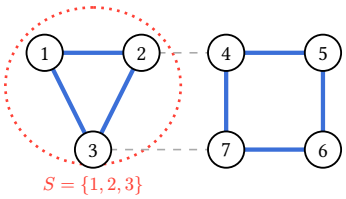


4 lati su 4 vertici, e il grafo è anche connesso, ma non è un giro: il vertice 3 tocca **tre** lati scelti, il 4 uno solo.

Nel giro invece in ogni città **entro una volta ed esco una volta**: ogni vertice tocca esattamente **due** lati scelti (ha **grado** 2). Fissato il vertice i, sommo le x dei lati che lo toccano:

$$\sum_{j:\{i,j\}\in E} x_{ij} = 2 \quad \forall i \in V$$

Questo però non basta. I lati in blu qui sotto danno a ogni vertice esattamente due lati, eppure non sono **un** giro ma **due** cicli separati (sottocicli):



Ogni vertice ha grado 2, ma il commesso viaggiatore non passa mai dal triangolo al quadrato.

Il rimedio sono i **vincoli di taglio dell'albero**: con $S = \{1, 2, 3\}$ nessun lato scelto esce da S ($\{2, 4\}$ e $\{3, 7\}$ hanno $x = 0$), quindi il vincolo è violato. Se il grafo è connesso e ogni vertice ha grado 2, l'unica possibilità è un solo ciclo che passa per tutti: un ciclo hamiltoniano.

$$\min z = \sum_{\{i,j\}\in E} c_{ij} x_{ij} \qquad \leftarrow \text{costo del giro}$$

$$\sum_{j:\{i,j\}\in E} x_{ij} = 2 \qquad \forall i \in V \qquad \leftarrow \text{entro ed esco da ogni città}$$

$$\sum_{i\in S, j\in V\setminus S} x_{ij} \geq 1 \qquad \forall S \subset V, S \neq \varnothing \qquad \leftarrow \text{niente sottocicli (connessione)}$$

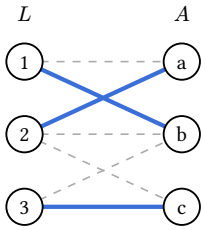
$$x_{ij} \in \{0, 1\} \qquad \forall \{i, j\} \in E$$

Il vincolo $\sum x_{ij} = n$ ora è **ridondante**: se ogni vertice tocca 2 lati, sommando su tutti i vertici conto $2n$, e ogni lato l'ho contato due volte (una per estremo), quindi i lati sono n.

> È il modello dell'albero di copertura più i vincoli di grado 2. Non è l'unico modello del TSP: si può usare una variabile per ogni ciclo hamiltoniano (ma sono un numero esponenziale e bisogna generarli tutti), oppure sostituire i vincoli di taglio con vincoli che vietano direttamente i sottocicli.

1.4 Assegnamento, semi-assegnamento e colorazione di grafi

Un'azienda deve assegnare delle attività a dei lavoratori. Lavoratori L e attività A sono due insiemi con lo stesso numero n di elementi. Assegnare il lavoratore $i \in L$ all'attività $j \in A$ costa c_{ij}. Tutte le attività vanno assegnate e ogni lavoratore riceve **una e una sola** attività; si vuole il costo minimo.



Si può vedere come una scelta di lati sul grafo **bipartito** $G = (L \cup A, E)$: a sinistra i lavoratori, a destra le attività, un lato per ogni coppia (nel disegno solo alcuni).

Un assegnamento è un insieme di lati in cui ogni vertice, di qualunque lato, tocca **esattamente un** lato scelto (in blu: 1 fa b, 2 fa a, 3 fa c).

$$x_{ij} = \begin{cases} 1 \text{ se assegno il lavoratore } i \in L \text{ all'attività } j \in A \\ 0 \text{ altrimenti} \end{cases}$$

$$\min z = \sum_{i \in L} \sum_{j \in A} c_{ij} x_{ij} \qquad \leftarrow \text{costo totale}$$

$$\sum_{j \in A} x_{ij} = 1 \quad \forall i \in L \qquad \leftarrow \text{ogni lavoratore ha una sola attività}$$

$$\sum_{i \in L} x_{ij} = 1 \quad \forall j \in A \qquad \leftarrow \text{ogni attività ha un solo lavoratore}$$

$$x_{ij} \in \{0, 1\} \quad \forall i \in L, j \in A$$

Ogni riga è il vincolo «esattamente uno fra…» delle relazioni logiche ($x_A + x_B = 1$). Servono **tutte e due**: con solo «ogni attività ha un lavoratore» potrei dare due attività allo stesso lavoratore e lasciarne un altro senza niente; con solo l'altra, due lavoratori potrebbero fare la stessa attività. Insieme si chiamano **vincoli di assegnamento**. L'assegnamento è un **problema polinomiale**.

> **Da sapere — problema polinomiale** (aggiunto, non spiegato in aula)
> Un problema è **polinomiale** se esiste un algoritmo che lo risolve in un tempo che cresce come una potenza della dimensione (n^2, n^3, …) e non come 2^n. In pratica: si risolve in fretta anche quando n è grande.

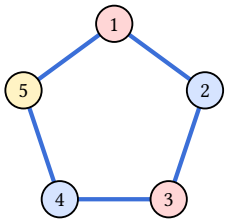
Gli stessi vincoli tornano nel commesso viaggiatore su un grafo **orientato**, dove il lato (i, j) va da i a j e non è lo stesso di (j, i). Il «grado 2» si spezza in due vincoli simili a quelli dell'assegnamento: da ogni città **esco** una volta, $\sum_j x_{ij} = 1$, e in ogni città **entro** una volta, $\sum_i x_{ij} = 1$.

Nell'assegnamento «esattamente uno» vale **dai due lati**. Se vale da **un lato solo** si parla di **vincoli di semi-assegnamento**: ogni elemento del primo insieme riceve esattamente un elemento del secondo, ma un elemento del secondo può essere dato a molti (o a nessuno).

Assegnamento di frequenze. Una compagnia telefonica copre la città con un insieme di antenne $S = \{1, \ldots, n\}$, tutte attive. A ogni antenna va assegnata una frequenza fra quelle disponibili $F = \{f_1, \ldots, f_m\}$. Due antenne troppo vicine non possono avere la stessa frequenza, altrimenti interferiscono. Ogni

frequenza ha un costo (lo stesso per tutte), quindi si vuole coprire la città usando il **minor numero di frequenze**.

L'interferenza si disegna con il **grafo di incompatibilità** $G = (S, E)$: i vertici sono le antenne e c'è il lato $\{i, j\} \in E$ se le antenne $i \neq j$ sono troppo vicine.



Ogni frequenza è un **colore**: **rosso** f_1, **blu** f_2, **giallo** f_3. Due antenne unite da un lato devono avere colori diversi. Le antenne 1 e 3 non sono vicine e possono condividere f_1: è il semi-assegnamento, una frequenza serve più antenne.

Qui servono 3 frequenze: con due sole, alternandole lungo il giro 1-2-3-4-5, la 5 e la 1 (vicine) avrebbero la stessa.

Variabili: due famiglie, una per decidere **chi usa cosa**, una per sapere **quali frequenze sono usate**:

$$x_{if} = \begin{cases} 1 \text{ se assegno la frequenza } f \in F \text{ all'antenna } i \in S \\ 0 \text{ altrimenti} \end{cases} \qquad y_f = \begin{cases} 1 \text{ se uso la frequenza } f \in F \\ 0 \text{ altrimenti} \end{cases}$$

$$\begin{aligned} \min z = \sum_{f \in F} y_f & & & \leftarrow \text{numero di frequenze usate} \\ \sum_{f \in F} x_{if} = 1 & & i \in S & \leftarrow \text{ogni antenna ha una sola frequenza} \\ x_{if} + x_{jf} \leq 1 & & \{i, j\} \in E, f \in F & \leftarrow \text{antenne vicine: frequenze diverse} \\ y_f \geq x_{if} & & i \in S, f \in F & \leftarrow \text{una frequenza assegnata è una frequenza usata} \\ x_{if} \in \{0, 1\} & & i \in S, f \in F & \\ y_f \in \{0, 1\} & & f \in F & \end{aligned}$$

Ogni vincolo è una relazione logica già vista:

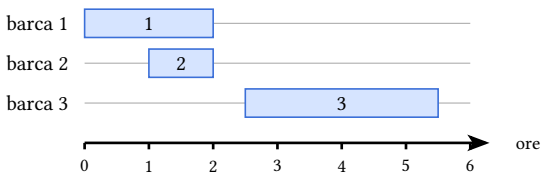
- $\sum_f x_{if} = 1$ è «**esattamente uno**»: il semi-assegnamento (per ogni antenna, non per ogni frequenza);
- $x_{if} + x_{jf} \leq 1$ è «**al massimo uno**»: fra due antenne vicine i e j al massimo una usa f. Si scrive per ogni lato del grafo di incompatibilità e per ogni frequenza.
- $y_f \geq x_{if}$ è il **legame** fra le due famiglie di variabili: l'implicazione «se assegno f all'antenna i, allora f è usata» ($x_{if} \leq y_f$). Se nessuna antenna usa f, y_f sarebbe libera, ma il min la porta a 0.

Senza quest'ultimo vincolo il modello sarebbe sbagliato: niente lega le y alle x, quindi il min metterebbe tutte le y_f a 0 (costo 0) mentre le antenne usano comunque le frequenze.

> Ogni volta che un modello ha **due o più famiglie di variabili** serve un vincolo che le leghi. Il significato che do a una variabile quando la definisco a parole («y_f vale 1 se uso la frequenza») è come un commento nel codice: il solutore (il programma che risolve il modello) non lo legge. Quel significato deve essere imposto dai vincoli.

Questo modello è la **colorazione di un grafo** (graph coloring): dare un colore a ogni vertice, con il minor numero di colori, in modo che due vertici uniti da un lato abbiano colori diversi. Lo stesso modello risolve anche il prossimo problema.

Moli e barche (ordinamento di lavori su macchine). Un porto ha un insieme di moli $M = \{1, ..., k\}$, tutti uguali. Ogni giorno arrivano n barche, $B = \{1, ..., n\}$. La barca i arriva all'istante $t_i \geq 0$ e occupa il molo per una durata d_i per scaricare, quindi riparte a $t_i + d_i$. Le barche non possono aspettare: vanno servite appena arrivano. Ogni barca va assegnata a **esattamente un molo**, e due barche non possono stare sullo stesso molo in intervalli di tempo che si sovrappongono. Si vuole usare il **minor numero di moli**.



Esempio con tre barche: la 1 sta in porto nell'intervallo $[0, 2]$, la 2 in $[1, 2]$, la 3 in $[2.5, 5.5]$.

La 1 e la 2 si sovrappongono (quando arriva la 2, la 1 è ancora al molo): non possono condividere un molo. La 3 arriva quando le altre due sono già ripartite: può usare il molo di una delle due. Bastano 2 moli.

Viene da pensare a una variabile per ogni molo e ogni istante («il molo m è occupato all'istante t»). Non serve: **il tempo qui non è una decisione**. Arrivo e durata sono dati, e una variabile serve solo per ciò che devo decidere (servirebbe se potessi scegliere **quando** servire una barca). In più gli istanti sono infiniti: bisognerebbe spezzare il tempo in intervallini (**discretizzare**).

Il tempo si usa **prima** di scrivere il modello, sui dati, per costruire il **grafo di incompatibilità** $G = (B, E)$: i vertici sono le barche, e c'è un lato fra due barche quando i loro intervalli hanno almeno un istante in comune,

$$\{i, j\} \in E \quad \text{se} \quad [t_i, t_i + d_i] \cap [t_j, t_j + d_j] \neq \varnothing$$

($\cap$ è l'intersezione, $\varnothing$ l'insieme vuoto: «l'intersezione non è vuota»). Nell'esempio E ha un solo lato, $\{1, 2\}$.

Le decisioni sono due, come per le antenne: **su quale molo va ogni barca** e **quali moli uso**.

$$x_{im} = \begin{cases} 1 \text{ se assegno la barca } i \in B \text{ al molo } m \in M \\ 0 \text{ altrimenti} \end{cases} \qquad y_m = \begin{cases} 1 \text{ se uso il molo } m \in M \\ 0 \text{ altrimenti} \end{cases}$$

$$\begin{aligned} \min z = \sum_{m \in M} y_m & & & \leftarrow \text{numero di moli usati} \\ \sum_{m \in M} x_{im} = 1 & & i \in B & \leftarrow \text{ogni barca ha un solo molo} \\ x_{im} + x_{jm} \leq 1 & & \{i, j\} \in E, m \in M & \leftarrow \text{barche incompatibili: moli diversi} \\ y_m \geq x_{im} & & i \in B, m \in M & \leftarrow \text{un molo con una barca è un molo usato} \\ x_{im} \in \{0, 1\} & & i \in B, m \in M & \\ y_m \in \{0, 1\} & & m \in M & \end{aligned}$$

È **lo stesso modello** dell'assegnamento di frequenze, con altri nomi:

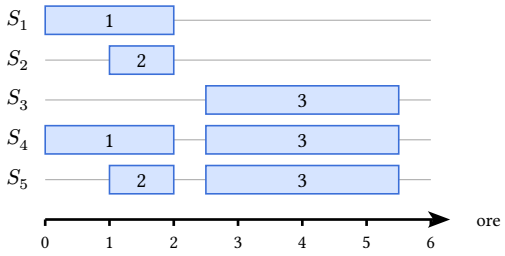
	frequenze	**moli e barche**	**lavori su macchine**
vertici del grafo	antenne	barche	lavori
colori	frequenze	moli	macchine
lato = incompatibili	troppo vicine	intervalli sovrapposti	lavori nello stesso momento

Prima lo stesso problema aveva modelli diversi; qui succede il contrario: **lo stesso modello risolve problemi diversi**.

1.5 Selezione di sottoinsiemi: partizione, copertura, riempimento

Nel modello dei moli la soluzione è costruita a pezzettini: una variabile per ogni coppia barca-molo. Niente vieta di usare **pezzi più grandi**. Invece di decidere barca per barca, elenco tutti i **turni** possibili di un molo (cioè tutti i gruppi di barche che un molo può servire in una giornata senza sovrapposizioni) e uso una variabile per ogni turno: lo scelgo oppure no.

Sia $\mathcal{F}$ la famiglia (un «insieme di insiemi») di tutti i sottoinsiemi non vuoti $S \subseteq B$ di barche che **non si sovrappongono fra loro**: ogni $S \in \mathcal{F}$ è un possibile turno (o schedulazione) di un molo. Con le tre barche dell'esempio i turni possibili sono 5:



$$\mathcal{F} = \{\{1\}, \{2\}, \{3\}, \{1,3\}, \{2,3\}\}$$

Un molo può servire una barca sola, oppure la 1 e poi la 3, oppure la 2 e poi la 3. Il turno $\{1, 2\}$ **non c'è**: le barche 1 e 2 sono incompatibili. L'incompatibilità è già sistemata quando costruisco $\mathcal{F}$, sui dati, e nel modello non compare più.

I moli sono tutti uguali: non serve dire **quale** molo fa un turno, basta sapere quali turni vengono fatti.

Variabili: una per turno,

$$y_S = \begin{cases} 1 \text{ se uso un molo con il gruppo di barche } S \in \mathcal{F} \\ 0 \text{ altrimenti} \end{cases}$$

Funzione obiettivo: ogni turno scelto occupa un molo, quindi i moli usati sono $\sum_{S \in \mathcal{F}} y_S$, da minimizzare.

Vincoli: ogni barca deve essere servita, e una volta sola. La barca 1 compare nei turni S_1 e S_4: fra questi due ne devo scegliere esattamente uno. Stesso discorso per le altre:

$$y_1 + y_4 = 1 \quad \text{(barca 1)} \qquad y_2 + y_5 = 1 \quad \text{(barca 2)} \qquad y_3 + y_4 + y_5 = 1 \quad \text{(barca 3)}$$

In generale, per ogni barca i sommo le variabili dei soli turni che la contengono:

$$\begin{aligned} \min z = \sum_{S \in \mathcal{F}} y_S & \qquad \leftarrow \text{numero di moli usati} \\ \sum_{S \in \mathcal{F} : i \in S} y_S = 1 \quad \forall i \in B & \qquad \leftarrow \text{ogni barca sta in un solo turno scelto} \\ y_S \in \{0, 1\} \qquad \forall S \in \mathcal{F} & \end{aligned}$$

Una soluzione è $y_2 = y_4 = 1$ e le altre a 0: un molo serve la barca 2, un altro serve la 1 e poi la 3. Due moli.

Lo stesso vincolo si scrive in modo più comodo con un **parametro binario**. Numero i turni con un insieme di indici $P = \{1, \dots, 5\}$ (y_p al posto di y_S) e definisco

$$a_{ip} = \begin{cases} 1 \text{ se la barca } i \in B \text{ è nel turno } p \in P \\ 0 \text{ altrimenti} \end{cases}$$

a_{ip}	$p=1$	$p=2$	$p=3$	$p=4$	$p=5$
$i=1$	**1**	0	0	**1**	0
$i=2$	0	**1**	0	0	**1**
$i=3$	0	0	**1**	**1**	**1**

Ogni colonna è un turno, ogni riga una barca. Il vincolo della barca i si legge sulla riga i: moltiplico ogni y_p per il numero sopra e sommo. Dove c'è 0 la variabile sparisce dalla somma, dove c'è 1 resta.

Riga 1: $1y_1 + 0y_2 + 0y_3 + 1y_4 + 0y_5 = 1$, cioè $y_1 + y_4 = 1$ come prima.

$$\min z = \sum_{p \in P} y_p \quad \text{s.t.} \quad \sum_{p \in P} a_{ip} y_p = 1 \quad \forall i \in B, \qquad y_p \in \{0, 1\} \quad \forall p \in P$$

> **Parametro o variabile?** a_{ip} è un **parametro**: un dato, come gli orari delle barche. Elencati i turni so già dove vale 1 e dove 0: non c'è niente da decidere, quindi non è fra le variabili e non ha un dominio. x_{im} nel modello di prima era una **variabile**: a quale molo va la barca lo decide il solutore.

> Il parametro può anche valere 2, 3, … per dire «quante volte», non solo «sì o no»: tornerà utile.

I due modelli a confronto: con le x_{im} ho poche variabili (barche × moli) e molti vincoli; con i turni ho **pochissimi vincoli** (uno per barca) ma **tantissime variabili**, una per ogni gruppo compatibile, e i gruppi possono essere un numero esponenziale. Questi modelli funzionano molto bene quando le variabili si generano un po' alla volta invece che tutte insieme, ma è una tecnica che nel corso non si vede.

Scegliere turni in modo che ogni barca stia in **esattamente uno** vuol dire dividere B in gruppi che non si sovrappongono e non lasciano fuori nessuno: una **partizione** di B. Cambiando il segno del vincolo si ottengono tre problemi con un nome:

problema	**per ogni $i \in B$**	**ogni elemento sta in…**	**per le barche**
partizione set partitioning	$\sum_{S \in \mathcal{F} : i \in S} y_S = 1$	esattamente un insieme scelto	ogni barca a un solo molo
copertura set covering	$\sum_{S \in \mathcal{F} : i \in S} y_S \geq 1$	almeno un insieme scelto	ogni barca ad almeno un molo
riempimento set packing	$\sum_{S \in \mathcal{F} : i \in S} y_S \leq 1$	al più un insieme scelto	ogni barca al più a un molo

> I nomi sono nomenclatura: il prof dice che non li chiederà («e poi mi smentirò»). Servono come campanello: se un problema è una partizione, una copertura o un riempimento, il modello c'è già.

Selezione di una configurazione: l'home restaurant. All'esame e al compitino l'esercizio di modellazione parte da un **testo**, come questo. Tommaso ha un home restaurant e sabato sera ha una prenotazione. La mattina va al mercato con **16,5 €**. Gli ingredienti si vendono solo in **casse** già fatte, che non si possono dividere:

Cassa Proteica (M) **6,5 €**	**Cassa Orto** (O) **4,0 €**	**Barattolo Salse** (S) **1,5 €**
1 filetto M_f 1 avanzo / ossa M_a	1 foglie miste O_i 1 radici O_r	1 salsa S

Deve decidere **quante casse comprare** e **quali piatti mettere nel menù**. Ogni piatto usa alcuni ingredienti (una porzione ciascuno) e ha un **valore** v_c, cioè quanto piace ai clienti. Tommaso vuole il menù di valore totale massimo. In più, se il menù ha **almeno 4 piatti** guadagna un **bonus di 10 punti** (menù vario).

c	**piatto**	M_f	M_a	O_i	O_r	S	valore v_c
1	Tagliata con insalata	**1**	**1**	**1**	0	0	6
2	Stufato rustico	**1**	**1**	0	0	**1**	3
3	Filetto al sale	**1**	0	0	0	0	2
4	Insalata di carne	**1**	0	**1**	0	**1**	5
5	Filetto con contorno	**1**	**1**	**1**	**1**	0	9
6	Stufato classico	**1**	**1**	0	0	0	10
7	Salsa del giorno	0	0	0	0	**1**	7
8	Piatto gourmet	**1**	**1**	**1**	**1**	**1**	8

La tabella piatto-ingrediente è una matrice di 0 e 1: è di nuovo un parametro binario.

Variabili. Le decisioni sono tre:

- quali piatti fare: $x_c \in \{0,1\}$ per ogni piatto $c \in C = \{1,\dots,8\}$, vale 1 se il piatto c è nel menù;
- quante casse comprare: y_M, y_O, y_S. Se ne può comprare più di una, quindi non sono binarie ma **intere**, $y \in \mathbb{N}_0$ (0, 1, 2, ...): dicono **se** compro e **quante**;
- se prendo il bonus: $z \in \{0,1\}$, vale 1 se il menù ha almeno 4 piatti.

Funzione obiettivo: il valore di ogni piatto scelto, più il bonus $b = 10$ se $z = 1$.

$$\max w = \sum_{c \in C} v_c x_c + bz \qquad \text{cioè} \quad \max 6x_1 + 3x_2 + 2x_3 + 5x_4 + 9x_5 + 10x_6 + 7x_7 + 8x_8 + 10z$$

Vincolo del bonus. Così com'è, z è slegata dal resto: il max la metterebbe sempre a 1 e prenderebbe il bonus anche con un piatto solo. È la situazione delle y_f con le antenne: serve un vincolo che leghi z alle x_c. Deve impedire $z = 1$ quando i piatti sono meno di 4:

$$\sum_{c \in C} x_c \geq 4z \qquad \text{(o, che è lo stesso,} \quad z \leq \frac{1}{4} \sum_{c \in C} x_c \ \text{)}$$

Con $z = 0$ dice $\sum x_c \geq 0$, sempre vero. Con $z = 1$ dice $\sum x_c \geq 4$: posso avere $z = 1$ solo con almeno 4 piatti. Visto dall'altra parte: con 3 piatti $z \leq \frac{3}{4}$, e una binaria $\leq \frac{3}{4}$ può solo valere 0.

E quando i piatti sono 4 o più? Il vincolo permette sia $z = 0$ sia $z = 1$. Ma z ha coefficiente positivo ($+10$) in una funzione obiettivo di **massimo**: appena può, il modello la mette a 1. **Basta questo vincolo** perché la funzione obiettivo spinge dalla parte giusta.

Se z non fosse nella funzione obiettivo (o il suo coefficiente spingesse dalla parte sbagliata), niente la obbligherebbe a valere 1 con 4 piatti o più. Per avere «$z = 1$ **se e solo se** i piatti sono almeno 4» serve un secondo vincolo, che obbliga z a 1:

$$\sum_{c \in C} x_c \leq 3 + (|C| - 3)z$$

($|C| = 8$ è il numero di piatti.) Con $z = 0$ dice $\sum x_c \leq 3$: per fare 4 piatti o più devo per forza avere $z = 1$. Con $z = 1$ dice $\sum x_c \leq |C|$, sempre vero. La tabella mostra, per ogni numero di piatti nel menù, quali valori di z lascia passare ciascun vincolo:

piatti nel menù	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
$\sum x_c \geq 4z$	**0**	**0**	**0**	**0**	0 o 1	0 o 1	0 o 1	0 o 1	0 o 1
$\sum x_c \leq 3 + 5z$	0 o 1	0 o 1	0 o 1	0 o 1	**1**	**1**	**1**	**1**	**1**
tutti e due	**0**	**0**	**0**	**0**	**1**	**1**	**1**	**1**	**1**

Il primo vincolo è del tipo «**posso** solo se» (tiene z a 0), il secondo del tipo «**devo** se» (spinge z a 1): lo stesso schema delle relazioni logiche.

Vincoli sugli ingredienti. I piatti scelti non possono usare più ingredienti di quelli comprati. Ogni Cassa Proteica contiene un filetto, quindi i filetti disponibili sono y_M; quelli che servono sono tanti quanti i piatti scelti che usano il filetto. Un vincolo per ogni ingrediente, letto dalle colonne della tabella:

$$\begin{aligned}
&\text{filetto } M_f\text{:} && x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_8 \ \leq\ y_M \\
&\text{avanzo / ossa } M_a\text{:} && x_1 + x_2 + x_5 + x_6 + x_8 \ \leq\ y_M \\
&\text{foglie } O_i\text{:} && x_1 + x_4 + x_5 + x_8 \ \leq\ y_O \\
&\text{radici } O_r\text{:} && x_5 + x_8 \ \leq\ y_O \\
&\text{salsa } S\text{:} && x_2 + x_4 + x_7 + x_8 \ \leq\ y_S
\end{aligned}$$

Se faccio i piatti 5 e 6, tutti e due con il filetto, il primo vincolo dà $y_M \geq 2$: due casse proteiche. Può capitare di comprare ingredienti che poi non uso (con le casse arrivano anche quelli).

Scrivere cinque vincoli a mano va bene qui, ma non con cento ingredienti. La stessa cosa si scrive in forma generale in due modi. Con gli **insiemi**: chiamo $H = \{M_f, M_a, O_i, O_r, S\}$ gli ingredienti e C_h l'insieme dei piatti che usano l'ingrediente h; per il filetto, $\sum_{c \in C_{M_f}} x_c \leq y_M$. Oppure con il **parametro binario** a_{hc}, che vale 1 se il piatto c usa l'ingrediente h (è la tabella dei piatti): $\sum_{c \in C} a_{M_f,c} x_c \leq y_M$. La somma è su tutti i piatti, e quelli che non usano il filetto spariscono perché moltiplicati per 0.

Il parametro permette di arrivare a **una sola formula per tutti gli ingredienti**. Chiamo $J = \{M, O, S\}$ i tipi di cassa e aggiungo un secondo parametro, $d_{hj} = 1$ se l'ingrediente h sta nella cassa j (0 altrimenti):

$$\underbrace{\sum_{c \in C} a_{hc} x_c}_{\text{quanto ne serve}} \leq \underbrace{\sum_{j \in J} d_{hj} y_j}_{\text{quanto ne compro}} \qquad \forall h \in H$$

> Questa forma regge anche casi che le altre non reggono, cambiando solo i numeri nei parametri: un piatto che usa **due** porzioni dello stesso ingrediente ($a_{hc} = 2$), una cassa che contiene **più unità** di un ingrediente ($d_{hj} = 2$), lo stesso ingrediente presente in **casse diverse** (più d_{hj} a 1 sulla stessa riga). All'esame va bene anche la forma lunga.

Vincolo di budget, come nello zaino: con k_j costo della cassa j e $B = 16.5$,

$$\sum_{j \in J} k_j y_j \leq B \qquad \text{cioè} \quad 6.5 y_M + 4 y_O + 1.5 y_S \leq 16.5$$

Il $\leq$ perché posso spendere anche meno. Non serve una variabile in più per «quanto spendo»: è già la somma a sinistra.

Il modello completo, con i domini (che vanno sempre scritti: il solutore non sa da solo che z è binaria):

$$\max w = \sum_{c \in C} v_c x_c + bz \qquad \leftarrow \text{valore del menù più bonus}$$

$$\sum_{c \in C} x_c \geq 4z \qquad \leftarrow \text{bonus solo con almeno 4 piatti}$$

$$\sum_{c \in C} a_{hc} x_c \leq \sum_{j \in J} d_{hj} y_j \qquad \forall h \in H \quad \leftarrow \text{ingredienti usati} \leq \text{comprati}$$

$$\sum_{j \in J} k_j y_j \leq B \qquad \leftarrow \text{budget}$$

$$x_c \in \{0,1\} \quad \forall c \in C, \quad y_j \in \mathbb{N}_0 \quad \forall j \in J, \quad z \in \{0,1\}$$

Instradamento: una variabile per ogni cammino. Nei moli la famiglia $\mathcal{F}$ si poteva elencare guardando gli orari. Spesso la famiglia non è data: va costruita dalle regole del testo. La Trans-Port, azienda di logistica, deve pianificare i viaggi dei suoi camion. Ogni camion parte dal **deposito** d, **consegna** container pieni agli **importatori** (insieme I) e/o **ritira** container vuoti dagli **esportatori** (insieme E), poi torna a d. Le regole:

- un camion porta **al massimo due container**;
- **prima consegna, poi ritira**: mai un esportatore prima di un importatore;
- si conosce la distanza c_{hk} fra ogni coppia di punti (deposito, importatori, esportatori), uguale nei due versi;
- ogni importatore i ha una domanda q_i di container da ricevere, ogni esportatore e una domanda q_e di container da far ritirare;

- i camion sono quanti ne servono (un camion rientrato può ripartire).

Si vogliono soddisfare tutte le domande percorrendo la **minima distanza totale**.

Con due container al massimo e la regola «prima consegno, poi ritiro», i giri possibili sono pochi: al più 2 fermate dagli importatori seguite da al più 2 dagli esportatori. Si dividono per numero di fermate:

insieme	cammini	perché non ce ne sono altri
P_1 1 fermata	d → i → d d → e → d	
P_2 2 fermate	d → i_1 → i_2 → d d → i → e → d d → e_1 → e_2 → d	manca $d \to e \to i \to d$: prima si consegna, poi si ritira
P_3 3 fermate	d → i_1 → i_2 → e → d d → i → e_1 → e_2 → d	mai tre importatori o tre esportatori: il camion porta due container
P_4 4 fermate	d → i_1 → i_2 → e_1 → e_2 → d	parte con 2 pieni, torna con 2 vuoti

L'insieme di tutti i cammini è $P = P_1 \cup P_2 \cup P_3 \cup P_4$. Da qui in poi tutto il lavoro è sui dati, prima del modello:

- il **costo** c_p del cammino p è la somma delle distanze dei suoi tratti. Per il cammino $d \to i \to e \to d$ è $c_{di} + c_{ie} + c_{ed}$. È un parametro: lo calcolo una volta generati i cammini;
- nulla vieta $i_1 = i_2$ (o $e_1 = e_2$): lo stesso camion porta **due container allo stesso cliente**, e il tratto fra le due «fermate» costa 0. Per questo il parametro che dice se un cammino passa da un cliente non è binario: a_{pi} = numero di volte (0, 1 o 2) che il cammino p visita l'importatore i, e a_{pe} lo stesso per l'esportatore e. Ecco il caso in cui il parametro vale 2;
- se un cliente è sia importatore sia esportatore, lo si tratta come due punti diversi a distanza 0.

Variabili: y_p = numero di volte che viene fatto il cammino $p \in P$. È intera e non binaria perché lo stesso giro può servire più volte (un importatore che aspetta 6 container riceve più camion).

$$\min \sum_{p \in P} c_p y_p \qquad \leftarrow \text{distanza totale}$$

$$\sum_{p \in P} a_{pi} y_p = q_i \quad \forall i \in I \qquad \leftarrow \text{ogni importatore riceve i suoi container}$$

$$\sum_{p \in P} a_{pe} y_p = q_e \quad \forall e \in E \qquad \leftarrow \text{da ogni esportatore ritiro i suoi container}$$

$$y_p \in \mathbb{N}_0 \qquad \forall p \in P$$

I vincoli hanno la forma della partizione, con q_i al posto di 1: sommo i cammini che passano da i, ciascuno contato quanti container gli lascia, e il totale deve essere la sua domanda. La capacità del camion e l'ordine consegna-ritiro **non compaiono nel modello**: sono già dentro la costruzione di P, come l'incompatibilità delle barche era dentro $\mathcal{F}$.

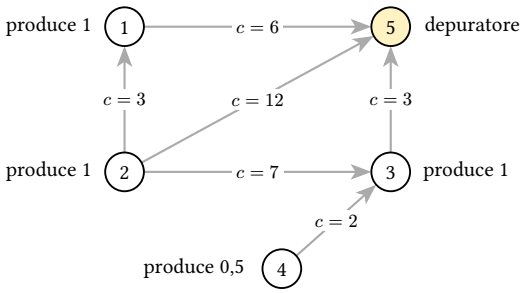
Tutta la difficoltà sta nel generare i cammini; il modello poi è semplice. Qui si può fare perché i cammini hanno al più 4 fermate: con giri lunghi sarebbero troppi, e si generano solo quando servono (generazione di colonne, non si fa nel corso).

1.6 Problemi di flusso su rete

Le variabili **binarie** dicono se faccio una cosa, le **intere** quante volte. Anche una variabile **continua** (un numero reale) può dire sì o no: se vale 0 la scelta non è fatta, se è positiva è fatta e il valore dice **quanto**.

Si usa nei **problemi di flusso su rete**. Una rete è un grafo orientato: qualcosa (merce, energia, dati, acqua, gas) si muove lungo gli archi, e le variabili dicono **quanto** ne passa su ogni arco, quindi anche quali archi si usano.

Il problema delle fognature. Una città ha 4 quartieri, i nodi 1, 2, 3 e 4. Ogni quartiere produce una quantità nota di acque reflue, in m³/h: 1 i quartieri 1, 2 e 3, e 0,5 il quartiere 4. Tutta l'acqua va portata a un unico depuratore, il nodo 5. Le condotte che si possono costruire sono gli archi $A = \{(1,5),(2,1),(2,3),(2,5),(3,5),(4,3)\}$: ogni condotta ha un verso. Il costo della condotta (i,j) è proporzionale all'acqua che ci passa: c_{ij} per ogni m³/h. Si vuole decidere quali condotte costruire e quanto grandi, per portare tutto al depuratore al costo minimo. È un problema di **disegno di rete**.



Variabili: x_{ij} = flusso (m³/h) sulla condotta $(i,j) \in A$. È continua. Se $x_{ij} = 0$ la condotta non si costruisce; se è positiva si costruisce, e il valore è anche la sua **dimensione**.

Funzione obiettivo: ogni condotta costa c_{ij} per ogni unità di flusso,

$$\min \sum_{(i,j)\in A} c_{ij} x_{ij}$$

cioè $\min 6x_{15} + 3x_{21} + 7x_{23} + 12x_{25} + 3x_{35} + 2x_{43}$.

Vincoli. Senza vincoli il minimo sarebbe tutto a 0: niente condotte, costo zero. Bisogna dire dove va l'acqua. La regola è la **conservazione del flusso**: in ogni nodo l'acqua non si crea e non sparisce, quindi **tutto quello che entra è uguale a tutto quello che esce**. Quello che il quartiere produce conta come acqua che entra nel nodo.

nodo	entra		esce
1	$x_{21} + 1$	$=$	x_{15}
2	1	$=$	$x_{21} + x_{23} + x_{25}$
3	$x_{23} + x_{43} + 1$	$=$	x_{35}
4	0.5	$=$	x_{43}
5	$x_{15} + x_{25} + x_{35}$	$=$	$1 + 1 + 1 + 0.5 = 3.5$

Il nodo 4 non ha scelta: il suo 0,5 va tutto sulla condotta $(4,3)$. Il nodo 2 invece ha tre strade e deve dividere la sua unità fra x_{21}, x_{23} e x_{25}: è lì che il modello decide. Nel nodo 5 quello che «esce» è l'acqua che il depuratore assorbe: tutta quella prodotta, 3.5. Questo vincolo è **ridondante**: si ottiene sommando gli altri quattro.

Per scrivere questi vincoli in forma generale si dà un nome alla quantità che ogni nodo produce o richiede: il **deficit** b_i, cioè quanto **manca** al nodo i. Per ragioni storiche si ragiona «al contrario»: un nodo che produce ha deficit **negativo**.

deficit	**il nodo è…**	**nelle fognature**
$b_i < 0$	una **sorgente**: offre $\lvert b_i \rvert$ unità di flusso	i quartieri: $b_1 = b_2 = b_3 = -1$, $b_4 = -0.5$
$b_i > 0$	un **pozzo**: richiede b_i unità di flusso	il depuratore: $b_5 = 3.5$
$b_i = 0$	un **nodo di transito**: il flusso ci passa e basta	nessuno (sarebbe un incrocio di tubi)

Portando tutte le variabili a sinistra, «entra = esce» diventa **entra − esce = deficit**. Sono i **vincoli di conservazione del flusso** (o di bilancio), uno per nodo:

$$\sum_{j:(j,i)\in A} x_{ji} - \sum_{j:(i,j)\in A} x_{ij} = b_i \quad \text{per ogni nodo } i$$

La prima somma è sugli archi che **entrano** in i, la seconda su quelli che **escono**. Per il nodo 1: $x_{21} - x_{15} = -1$, che è $x_{21} + 1 = x_{15}$ di prima.

Tutto quello che le sorgenti offrono deve essere assorbito dai pozzi, quindi i deficit sommati fanno zero: $\sum_i b_i = 0$. Il deficit del depuratore non è un dato in più: $b_5 = -(b_1 + b_2 + b_3 + b_4) = 3.5$.

Resta il dominio: il flusso non può essere negativo, $x_{ij} \geq 0$ per ogni $(i, j) \in A$.

> Un nodo **di transito** è solo quello con $b_i = 0$. Se un nodo produce o richiede qualcosa ($b_i \neq 0$) non è di transito, anche se ci passa dentro il flusso di altri nodi: nelle fognature il 3 riceve l'acqua del 4 ma resta una sorgente.

Flusso di costo minimo (Minimum Cost Flow, MCF). Le fognature sono un caso di un problema più generale: un grafo orientato $G = (N, A)$, un deficit b_i per ogni nodo, un costo c_{ij} per unità di flusso su ogni arco e, in più, una **capacità** u_{ij}: il massimo flusso che l'arco può portare. Il modello è quello di prima con la capacità nel dominio:

$$\begin{aligned}
\min z = \sum_{(i,j)\in A} c_{ij} x_{ij} & & & \leftarrow \text{costo del flusso} \\
\sum_{j:(j,i)\in A} x_{ji} - \sum_{j:(i,j)\in A} x_{ij} = b_i & \quad \forall i \in N & & \leftarrow \text{conservazione del flusso} \\
0 \leq x_{ij} \leq u_{ij} & & \forall (i,j) \in A & \leftarrow \text{capacità e non negatività}
\end{aligned}$$

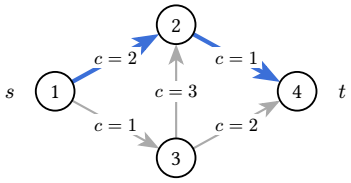
Quando un arco è pieno, il resto del flusso deve prendere altre strade. Le fognature sono un MCF senza capacità ($u_{ij} = +\infty$): lì la condotta si costruiva della misura giusta per il flusso.

Cammino di costo minimo (Shortest Path Problem, SPP). Su un grafo orientato con un costo c_{ij} per arco ci sono un nodo di partenza s e uno di arrivo $t \neq s$. Si cerca il cammino da s a t che costa meno, come fa Google Maps. Ci sono algoritmi fatti apposta, ma si può scrivere anche come MCF, con due piccole modifiche.

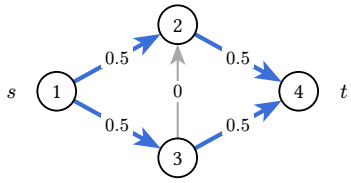
Un cammino è un flusso di **una sola unità**: sono io che mi sposto da s a t. Quindi s è una sorgente che offre 1, t un pozzo che chiede 1, e tutti gli altri nodi sono di transito: se il cammino ci passa entra 1 ed esce 1, altrimenti 0 e 0.

$$b_i = \begin{cases} -1 & i = s \\ 1 & i = t \\ 0 & \text{altrimenti} \end{cases}$$

Visto che passa una persona sola, la capacità si può mettere a 1 su tutti gli archi ($u_{ij} = 1$; va bene anche $+\infty$).



un cammino ottimo, $1 \to 2 \to 4$



mezzo cammino per parte: ammesso dal modello

Nell'esempio i cammini da 1 a 4 sono tre: $1 \to 2 \to 4$ costa $2+1=3$, $1 \to 3 \to 4$ costa $1+2=3$, $1 \to 3 \to 2 \to 4$ costa $1+3+1=5$. I primi due sono ottimi, ed è indifferente quale scegliere. Il modello:

nodo	**entra – esce**		b_i
1	$-x_{12} - x_{13}$	$=$	-1
2	$+x_{12} + x_{32} - x_{24}$	$=$	0
3	$+x_{13} - x_{32} - x_{34}$	$=$	0
4	$+x_{24} + x_{34}$	$=$	1

$$\min z = 2x_{12} + x_{13} + x_{24} + 3x_{32} + 2x_{34}$$

Nel nodo 1 esce soltanto, nel 4 entra soltanto. Nei nodi 2 e 3 quello che entra deve uscire.

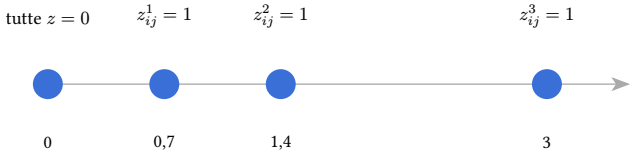
Il modello però ammette anche la soluzione a destra: $x_{12} = x_{24} = x_{13} = x_{34} = 0.5$, mezza persona su un cammino e mezza sull'altro. Rispetta tutti i bilanci (nel nodo 2 entra 0,5 ed esce 0,5) e costa $0.5 \cdot 3 + 0.5 \cdot 3 = 3$, come l'ottimo. Ma non è un cammino. Per avere un cammino vero serve il **vincolo di interezza**: $x_{ij} \in \{0, 1\}$.

> Se interessa solo il **valore** ottimo (quanto costa il cammino migliore), le variabili continue bastano: il valore esce giusto e il modello si risolve molto più in fretta. Per sapere **quale** cammino fare servono le binarie.

1.7 Variabili discrete, semicontinue e funzioni a tratti

A volte una variabile non può prendere tutti i valori di un intervallo, ma solo alcuni: pochi valori fissi, oppure zero o un intervallo, oppure deve seguire una funzione fatta a pezzi. Il trucco è sempre lo stesso: **una variabile binaria per ogni caso possibile**, che dice in quale caso sono.

Variabili a valori discreti. Nelle fognature x_{ij} era insieme il flusso e la dimensione della condotta. In realtà le condotte esistono solo in tre misure, con portata 0,7, 1,4 e 3: la condotta può essere più grande del flusso che ci passa. Servono due variabili per arco: x_{ij} resta il flusso, e y_{ij} è la dimensione della condotta, con $y_{ij} \in \{0, 0.7, 1.4, 3\}$ (0 = non la costruisco). Ora si paga la condotta, quindi il costo dipende dalla dimensione: $\min \sum_{(i,j)\in A} c_{ij} y_{ij}$.



Un dominio come $\{0, 0.7, 1.4, 3\}$ non si scrive direttamente: si usa una binaria per ogni valore, $z^h_{ij} = 1$ se la condotta (i, j) ha la misura h.

$$y_{ij} = 0.7z^1_{ij} + 1.4z^2_{ij} + 3z^3_{ij} \quad \forall (i,j) \in A \qquad \leftarrow \text{la dimensione è la misura scelta}$$

$$z^1_{ij} + z^2_{ij} + z^3_{ij} \le 1 \quad \forall (i,j) \in A \qquad \leftarrow \text{al massimo una misura}$$

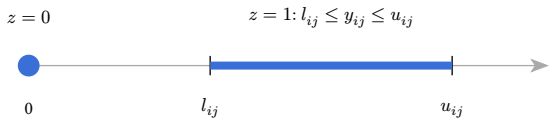
$$x_{ij} \le y_{ij} \quad \forall (i,j) \in A \qquad \leftarrow \text{il flusso non supera la portata}$$

$$z^h_{ij} \in \{0, 1\} \quad \forall (i,j) \in A, h = 1, 2, 3$$

Il ≤ 1 e non $= 1$ perché posso anche non scegliere nessuna misura: tutte le z a 0 danno $y_{ij} = 0$, e allora $x_{ij} \leq 0$, cioè niente condotta e niente flusso. Se lo 0 non fosse ammesso, cioè se dovessi per forza scegliere una delle misure, la somma sarebbe $= 1$. Il vincolo $x_{ij} \leq y_{ij}$ è il legame fra le due famiglie di variabili: senza, il min metterebbe tutte le y a 0.

In forma generale, con valori ammessi $d_1, \ldots, d_n$: $y = \sum_{i=1}^{n} d_i z_i$, $\sum_{i=1}^{n} z_i = 1$ (o ≤ 1 se è ammesso anche lo 0), $z_i \in \{0, 1\}$.

Variabili semicontinue. Ora la condotta, se la costruisco, può avere qualunque dimensione fra un minimo l_{ij} e un massimo u_{ij}. Quindi y_{ij} vale 0 oppure sta in $[l_{ij}, u_{ij}]$: non è un intervallo solo, c'è un buco fra 0 e l_{ij}.

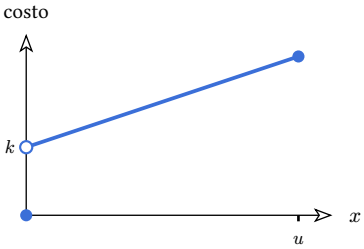


Con una binaria $z_{ij} = 1$ se costruisco la condotta, **moltiplico i due limiti per z**:

$$l_{ij} z_{ij} \leq y_{ij} \leq u_{ij} z_{ij} \qquad z_{ij} \in \{0, 1\}$$

Se $z = 1$ restano i limiti originali, $l \leq y \leq u$. Se $z = 0$ diventa $0 \leq y \leq 0$, cioè $y = 0$.

Costi di setup. Capita spesso nei problemi di produzione. Produrre costa c per ogni unità (costo variabile), ma avviare la produzione, per esempio accendere la macchina, costa k una volta sola, che faccia un pezzo o 500: è il **costo di setup**. In più, se produco, devo farlo fra una quantità minima l e una massima u. Con x = quantità prodotta e $y = 1$ se produco:



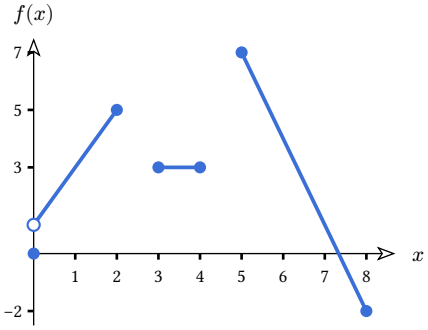
$$\begin{aligned} \min\ & cx + ky \\ & ly \leq x \leq uy \\ & x \geq 0, \quad y \in \{0, 1\} \end{aligned}$$

Il costo vale 0 se non produco e $k + cx$ se produco: nel disegno il salto in 0 è il setup.

È lo stesso trucco delle semicontinue: se $y = 0$ allora $x = 0$, e k non lo pago; se produco qualcosa il vincolo obbliga $y = 1$, e pago k. Una variabile dice **se** produco, l'altra **quanto**.

Se il testo non dà un massimo u, il legame fra x e y serve lo stesso. Allora il massimo me lo invento: un numero M **grande a piacere**, più grande di qualunque quantità possibile nel problema (se non si può superare 10, va bene 11 o 20). Con $l = 0$ il vincolo diventa solo $x \leq My$.

Funzioni lineari a tratti. Un costo può cambiare forma a pezzi: un prezzo fino a una certa quantità, poi uno sconto, poi un altro prezzo... Esempio:



$$f(x) = \begin{cases} 0 & x = 0 \\ 1 + 2x & x \in (0, 2] \\ 3 & x \in [3, 4] \\ 22 - 3x & x \in [5, 8] \end{cases}$$

Fra 2 e 3 e fra 4 e 5 la x non può stare: la funzione ha dei **buchi** (è discontinua).

Per scriverla nel modello devo sapere **in quale tratto** sono e **dove**, dentro quel tratto:

- una binaria per tratto, $z^i = 1$ se x sta nel tratto i. Non posso stare in due tratti insieme: $z^1 + z^2 + z^3 \leq 1$ (tutte a 0 vuol dire $x = 0$);
- una continua per tratto, w^i = il valore di x se sono nel tratto i, 0 altrimenti. Si lega alla sua z come una semicontinua: $3z^2 \leq w^2 \leq 4z^2$;
- $x = w^1 + w^2 + w^3$: al più una delle w è diversa da 0, ed è proprio la x;
- in ogni tratto f è una retta, termine noto + pendenza × x. Il termine noto è un **costo fisso**, pagato solo se sono in quel tratto (moltiplico per z^i); la pendenza è un **costo variabile** (moltiplico per w^i).

$$\begin{aligned} \min f = {} & z^1 + 2w^1 + 3z^2 + 22z^3 - 3w^3 && \leftarrow\ 1+2x,\ 3,\ 22-3x \text{ tratto per tratto} \\ & x = w^1 + w^2 + w^3 \\ & 0 \leq w^1 \leq 2z^1, \quad 3z^2 \leq w^2 \leq 4z^2, \quad 5z^3 \leq w^3 \leq 8z^3 \\ & z^1 + z^2 + z^3 \leq 1 \\ & x, w^1, w^2, w^3 \geq 0, \quad z^1, z^2, z^3 \in \{0, 1\} \end{aligned}$$

w^2 non compare in f perché il secondo tratto è piatto: pendenza 0. Se lo 0 non fosse fra i valori ammessi di x, la somma delle z sarebbe $= 1$.

> Il primo tratto è aperto in 0, $(0, 2]$, ma il $<$ stretto non si scrive: si mette $0 \leq w^1$. Di solito non cambia niente, perché si minimizza e fra $f = 1$ (primo tratto con $x = 0$) e $f = 0$ il modello sceglie 0. Se servisse davvero, si mette $\varepsilon z^1 \leq w^1$ con ε piccolo. All'esame non si chiede.

In generale, con n tratti $[l_i, u_i]$ che non si sovrappongono e $f = b_i + c_i x$ nel tratto i (la somma delle z è ≤ 1 se è ammesso lo 0, altrimenti $= 1$):

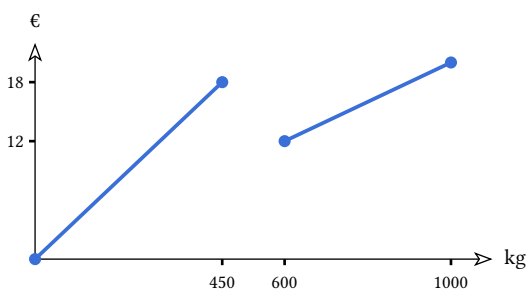
$$\min f = \sum_{i=1}^{n} (b_i z^i + c_i w^i) \quad \text{s.t.} \quad x = \sum_{i=1}^{n} w^i, \quad l_i z^i \leq w^i \leq u_i z^i, \quad \sum_{i=1}^{n} z^i \leq 1, \quad w^i \geq 0, \quad z^i \in \{0, 1\}$$

La fonderia con gli sconti. La fonderia del primo esempio deve sempre produrre 1000 kg, ma ora i tre materiali ferrosi (qui A, B e C, cioè x_1, x_2, x_3) si comprano da tre fornitori, ognuno con le sue condizioni:

fornitore	condizioni
A	fino a 350 kg 0,03 €/kg; da 350 a 750 kg 0,05 €/kg; oltre 750 kg 0,08 €/kg
B	fino a 600 kg 0,04 €/kg; oltre 600 kg 0,02 €/kg; ordini fra 450 e 600 kg non accettati
C	10 € fissi per qualunque quantità fra 100 e 400 kg; oltre 400 kg 0,06 €/kg; meno di 100 kg non si può

I vincoli sul silicio e sul manganese non cambiano. Cambia la funzione obiettivo: il costo di ogni materiale non è più prezzo × quantità, ma una funzione a tratti della quantità comprata.

Il caso più semplice è B. Tradotto: o ordino **da 0 a 450 kg a 0,04 €/kg**, o ordino **da 600 kg in su a 0,02 €/kg** (tutto al prezzo scontato). Oltre i 1000 kg non serve andare: in tutto ne servono 1000.



Due tratti, quindi due binarie e due continue. Il pedice è il materiale (2 = B), l'apice il tratto:

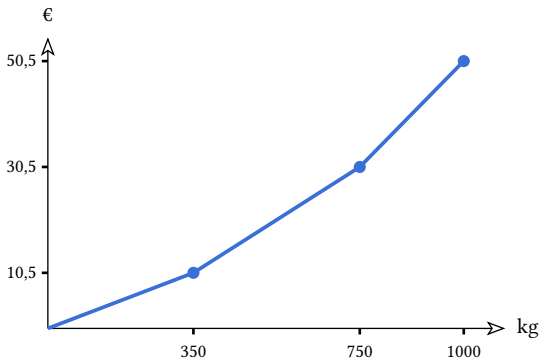
- $z_2^1 = 1$ se ordino fra 0 e 450 kg, w_2^1 = quanti kg a 0,04 €/kg;
- $z_2^2 = 1$ se ordino fra 600 e 1000 kg, w_2^2 = quanti kg a 0,02 €/kg.

$$
\begin{aligned}
& x_2 = w_2^1 + w_2^2 && \leftarrow \text{B comprato a uno dei due prezzi} \\
& 0 \le w_2^1 \le 450 z_2^1, \quad 600 z_2^2 \le w_2^2 \le 1000 z_2^2 && \leftarrow \text{ogni quantità nel suo intervallo} \\
& z_2^1 + z_2^2 \le 1, \quad z_2^1, z_2^2 \in \{0,1\} && \leftarrow \text{al massimo un prezzo (o niente)}
\end{aligned}
$$

e nella funzione obiettivo il termine $0.030 x_2$ diventa $0.04 w_2^1 + 0.02 w_2^2$.

Va bene anche contare in w_2^2 solo i kg **oltre** 600: $x_2 = w_2^1 + 600 z_2^2 + w_2^2$, con $0 \le w_2^2 \le (1000 - 600) z_2^2$ e costo $0.04 w_2^1 + 0.02 \cdot 600 z_2^2 + 0.02 w_2^2$. È lo stesso modello scritto in un altro modo.

Il materiale A ha i prezzi **a scaglioni**: i primi 350 kg costano 0,03 €/kg, i kg **successivi** fino a 750 costano 0,05 €/kg, quelli oltre 750 costano 0,08 €/kg. A differenza di B, comprarne di più non cambia il prezzo di tutto l'ordine: ogni scaglione paga il suo. Per esempio 800 kg costano $350 \cdot 0.03 + 400 \cdot 0.05 + 50 \cdot 0.08 = 34.5$ €.



Il costo totale è una funzione a tratti **senza buchi**, e ogni tratto è più ripido del precedente. Una binaria e una continua per scaglione, ma qui la continua conta solo i kg **dentro** quello scaglione:

- w_1^1 = kg comprati fra 0 e 350, w_1^2 = kg fra 350 e 750, w_1^3 = kg fra 750 e 1000;
- $z_1^h = 1$ se uso lo scaglione h.

$$
\begin{aligned}
& x_1 = w_1^1 + w_1^2 + w_1^3 && \leftarrow \text{A comprato = somma degli scaglioni} \\
& 0 \le w_1^1 \le 350 z_1^1, \quad 0 \le w_1^2 \le (750 - 350) z_1^2, \quad 0 \le w_1^3 \le (1000 - 750) z_1^3 \\
& z_1^1 \ge z_1^2 \ge z_1^3, \quad z_1^1, z_1^2, z_1^3 \in \{0,1\} && \leftarrow \text{uno scaglione si apre solo se è aperto il precedente}
\end{aligned}
$$

e nella funzione obiettivo il costo di A diventa $f_1 = 0.03 w_1^1 + 0.05 w_1^2 + 0.08 w_1^3$.

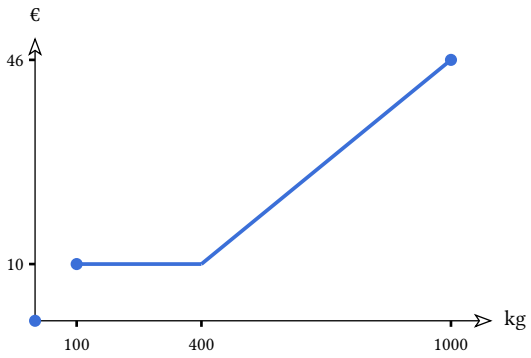
> Il modello non obbliga a **riempire** uno scaglione prima di passare al successivo: $w_1^1 = 0$, $w_1^2 = 400$ è ammesso. Non succede perché si minimizza e lo scaglione prima costa meno: il min lo riempie da solo.

Proprio per questo le binarie di A si possono togliere. Quando i prezzi **crescono** con la quantità ($0.03 \le 0.05 \le 0.08$) la funzione si dice **convessa**, e il min usa da solo gli scaglioni nell'ordine giusto, dal più economico:

$$
\begin{aligned}
& \min 0.03 w_1^1 + 0.05 w_1^2 + 0.08 w_1^3 \\
& \quad x_1 = w_1^1 + w_1^2 + w_1^3, \quad 0 \le w_1^1 \le 350, \quad 0 \le w_1^2 \le 750 - 350, \quad 0 \le w_1^3 \le 1000 - 750
\end{aligned}
$$

Conviene, perché le binarie rendono il problema molto più difficile da risolvere. Se invece i prezzi **calassero** con la quantità (funzione **concava**), il min riempirebbe per primo lo scaglione più economico, l'ultimo, saltando gli altri: lì le binarie servono.

Resta C: 10 € fissi per un ordine fra 100 e 400 kg; oltre 400 kg, 10 € più 0,06 €/kg per i kg oltre 400; meno di 100 kg non si può. Oppure non compro niente.



Due tratti, scritti come la seconda forma di B (la continua conta i kg **oltre** l'inizio del tratto):

- $z_3^1 = 1$ se ordino fra 100 e 400 kg, w_3^1 = kg oltre 100;
- $z_3^2 = 1$ se ordino fra 400 e 1000 kg, w_3^2 = kg oltre 400.

$$x_3 = 100z_3^1 + w_3^1 + 400z_3^2 + w_3^2$$

$$0 \le w_3^1 \le (400-100)z_3^1, \quad 0 \le w_3^2 \le (1000-400)z_3^2$$

$$z_3^1 + z_3^2 \le 1, \quad z_3^1, z_3^2 \in \{0,1\} \qquad \leftarrow \text{un tratto, l'“altro o niente}$$

$$f_3 = 10z_3^1 + 10z_3^2 + 0.06w_3^2 \qquad \leftarrow \text{costo di C}$$

Il 100 e il 400 entrano in x_3 moltiplicati per la binaria: con $z_3^1 = 1$ ho già 100 kg, il minimo ordinabile. La funzione obiettivo completa è $\min f_1 + f_2 + f_3 + 10x_4$, con f_2 il costo di B scritto prima e $10x_4$ il manganese puro, che non cambia.

1.8 Linearizzazioni: min-max, valore assoluto, vincoli disgiuntivi

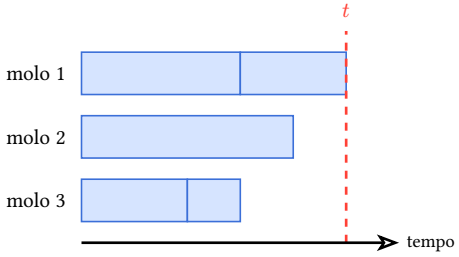
Finora ogni vincolo era lineare, o lo diventava con una binaria per caso. Restano tre cose che lineari non sono: il **massimo** fra più quantità, il **valore assoluto**, un **«oppure»** fra due vincoli. Per ognuna c'è un trucco.

Ordinamento di lavori su macchine: minimizzare il tempo di completamento. Una compagnia logistica ha n barche $B = \{1, \dots, n\}$, tutte già in rada, da assegnare ai moli $M = \{1, \dots, m\}$ per lo scarico. La barca i occupa il molo per un tempo p_i; un molo serve una barca alla volta, e le barche possono aspettare in rada finché vengono chiamate. Si vuole finire il prima possibile: minimizzare il **tempo di completamento massimo** (makespan), l'istante in cui termina l'ultimo scarico.

Rispetto ai moli di prima cambia tutto: i moli sono dati, le barche aspettano, e conta quando si finisce. Una barca per molo alla volta, senza buchi: il molo j finisce dopo $\sum_{i \in B} p_i x_{ij}$, la somma dei tempi delle barche che gli assegno ($x_{ij} = 1$ se la barca i va al molo j). Il makespan è il massimo fra i moli:

$$\min \max_{j \in M} \sum_{i \in B} p_i x_{ij}$$

Non è lineare. Il trucco: una variabile $t \ge 0$ che deve stare **sopra tutti** i tempi dei moli. (Si può anche dare un nome al tempo di fine di ogni molo, $T_j = \sum_i p_i x_{ij}$, e chiedere $t \ge T_j$: è lo stesso modello con variabili in più.) Se t sta sopra a tutti, sta sopra al più grande; il min la spinge giù finché tocca il più grande.



Ogni barra è un molo, divisa nelle sue barche. La linea rossa è t: deve stare a destra di tutte le barre, e al minimo coincide con la più lunga.

$$
\begin{aligned}
&\min t && && \leftarrow \text{makespan}\\
&\sum_{j\in M} x_{ij} = 1 && \forall i \in B && \leftarrow \text{ogni barca a un solo molo}\\
&\sum_{i\in B} p_i x_{ij} \le t && \forall j \in M && \leftarrow \text{ogni molo finisce entro } t\\
&x_{ij} \in \{0,1\} && \forall i \in B, j \in M, && t \ge 0
\end{aligned}
$$

Lo stesso trucco vale per qualunque **min-max** e **max-min**. Date delle funzioni lineari $f_i(x) = \sum_j a_{ij}x_j + b_i$, una per ogni $i \in F$, si aggiunge una variabile v:

	min-max	**max-min**
problema	$\min\{\max_{i\in F} f_i(x)\}$	$\max\{\min_{i\in F} f_i(x)\}$
diventa	$\min v \quad \text{s.t.} \quad v \ge f_i(x) \quad \forall i \in F$	$\max v \quad \text{s.t.} \quad v \le f_i(x) \quad \forall i \in F$

> Funziona solo in queste due combinazioni, dove la funzione obiettivo spinge v **contro** i vincoli. Con un max-max, $\max v$ e $v \ge f_i(x)$, niente ferma v: sale all'infinito.

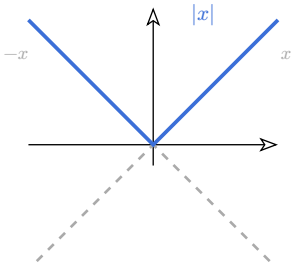
Equipartizione. Due fratelli devono spartirsi un'eredità di n beni indivisibili; il bene i vale a_i. Vogliono dividerli in due gruppi di valore il più vicino possibile: se S sono i beni del primo fratello,

$$
\min\left\{ \left| \sum_{i\in S} a_i - \sum_{i\notin S} a_i \right| : S \subseteq \{1,\dots,n\} \right\}
$$

Con $x_i = 1$ se il bene i va al fratello 1 e $x_i = 0$ se va al fratello 2, il primo riceve $\sum a_i x_i$ e il secondo $\sum a_i(1-x_i)$ (gli $x_i = 0$ diventano 1). Quindi

$$
\min z = \left| \sum_{i=1}^{n} a_i x_i - \sum_{i=1}^{n} a_i(1-x_i) \right|, \quad x_i \in \{0,1\}
$$

che non è lineare per via del valore assoluto.



Il valore assoluto è un massimo: $|x| = \max\{x, -x\}$. In blu $|x|$, che in ogni punto coincide con la più alta delle due rette tratteggiate.

Quindi minimizzare $|f(x)|$ è un min-max, e si sa scrivere:

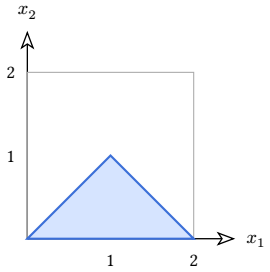
$$
\min|f(x)| = \min\max\{f(x), -f(x)\} \quad \to \quad \min v \quad \text{s.t.} \quad v \ge f(x), \quad v \ge -f(x)
$$

Per l'equipartizione:

$$
\begin{aligned}
&\min v\\
&v \ge \sum_{i=1}^{n} a_i x_i - \sum_{i=1}^{n} a_i(1-x_i)\\
&v \ge \sum_{i=1}^{n} a_i(1-x_i) - \sum_{i=1}^{n} a_i x_i\\
&x_i \in \{0,1\} \quad i = 1,\dots,n
\end{aligned}
$$

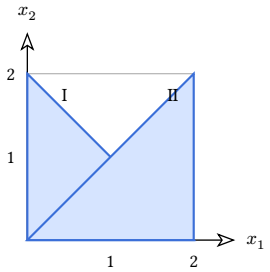
Vincoli disgiuntivi. In Programmazione Lineare l'insieme dei punti ammissibili è descritto da un numero finito di vincoli lineari $A_i x \le b_i$, $i = 1,\dots,m$ (A_i è la riga dei coefficienti del vincolo i). Valgono **tutti**

insieme: l'insieme è l'**intersezione** dei vincoli, e si chiama **poliedro convesso**. **Convesso** vuol dire che, presi due punti dentro, tutto il segmento che li unisce resta dentro.



Esempio: $x_1 + x_2 \leq 2$ (I), $-x_1 + x_2 \leq 0$ (II), $0 \leq x_1 \leq 2$, $0 \leq x_2 \leq 2$. Il punto deve stare sotto la retta di I **e** sotto quella di II: è il triangolo di vertici $(0,0)$, $(1,1)$, $(2,0)$.

Per il prof i vincoli disgiuntivi sono «la cosa più bella che può capitare» all'esame e all'orale. E se volessi «vale I **oppure** vale II»? L'insieme diventa l'**unione** dei due triangoli, e non è convesso: $(0,2)$ e $(2,2)$ stanno dentro, il punto in mezzo $(1,2)$ no. Con i soli vincoli lineari non si può scrivere.



Si usa una binaria per vincolo che dice se lo posso **violare**:

- $y_I = 1$ se posso violare $x_1 + x_2 \leq 2$, 0 altrimenti;
- $y_{II} = 1$ se posso violare $-x_1 + x_2 \leq 0$, 0 altrimenti;

e si vieta di violarli tutti e due.

$$
\begin{aligned}
&0 \leq x_1 \leq 2, \quad 0 \leq x_2 \leq 2 && \leftarrow \text{non cambiano} \\
&x_1 + x_2 \leq 2 + M_I y_I \\
&-x_1 + x_2 \leq 0 + M_{II} y_{II} \\
&y_I + y_{II} \leq 1, \quad y_I, y_{II} \in \{0,1\} && \leftarrow \text{non posso violarli entrambi}
\end{aligned}
$$

Con $y_I = 0$ il vincolo I è quello originale; con $y_I = 1$ il lato destro sale di M_I e, se M_I è abbastanza grande, il vincolo è sempre vero: è come se non ci fosse. Quanto grande? Quanto il massimo che il lato sinistro può superare il destro nel quadrato: $\max\{x_1 + x_2 - 2\} = 2$ e $\max\{-x_1 + x_2\} = 2$, quindi bastano $M_I = M_{II} = 2$. È il big-M dei costi di setup, usato per spegnere un vincolo.

In generale: ho dei vincoli $A_i x \leq b_i$, $i = 1, \ldots, m$, e dei sottoinsiemi di indici $S_1, \ldots, S_p$ (lo stesso vincolo può stare in più gruppi). Ogni gruppo descrive un poliedro convesso $X_h = \{x : A_i x \leq b_i, i \in S_h\}$, e voglio l'unione $X = X_1 \cup \ldots \cup X_p$, cioè «vale almeno un gruppo». Con una binaria y_h per gruppo, che vale 1 se **spengo** il gruppo h:

$$A_i x - M_h y_h \leq b_i \quad \forall i \in S_h, h = 1, \ldots, p \qquad y_1 + \ldots + y_p \leq p - 1 \qquad y_h \in \{0,1\}$$

Al massimo $p - 1$ gruppi spenti: almeno uno resta acceso, e x sta nel suo poliedro. Le costanti M_h si scelgono in modo che con $y_h = 1$ i vincoli del gruppo diventino ridondanti; si può sempre se gli X_h sono limitati (altrimenti si aggiungono dei limiti artificiali). Con le y_h si possono poi scrivere relazioni logiche fra i gruppi, come quelle fra i progetti.

La fonderia con le proprietà. Ancora la fonderia del primo esempio (1000 kg, tre materiali ferrosi e manganese puro, stessi dati), con due novità:

1. per essere venduto il ferro deve avere **almeno una** di tre proprietà: (i) almeno lo 0,45% di manganese; (ii) almeno il 3,25% di silicio; (iii) al più il 5,5% di silicio;
2. se ne ha **almeno due**, la fonderia prende un bonus di 127 € sul prezzo di vendita.

Si vuole il piano di produzione di costo minimo, al netto del bonus. I tre vincoli di qualità del modello originale ora non devono valere tutti: ne basta uno, è un vincolo disgiuntivo con tre gruppi di un vincolo ciascuno. Le binarie:

- y_i, y_{ii}, y_{iii} = 1 se il ferro ha la proprietà (i), (ii), (iii);
- $w = 1$ se le proprietà soddisfatte sono almeno due (prendo il bonus).

$$
\begin{aligned}
\min z = {} & 0.025x_1 + 0.030x_2 + 0.018x_3 + 10x_4 - 127w && \leftarrow \text{costo meno bonus} \\
& x_1 + x_2 + x_3 + x_4 = 1000 \\
& 45x_1 + 50x_2 + 40x_3 + 10000x_4 \geq 45000 - M(1 - y_i) && \leftarrow \text{(i) manganese minimo} \\
& 40x_1 + 10x_2 + 6x_3 \geq 32500 - M(1 - y_{ii}) && \leftarrow \text{(ii) silicio minimo} \\
& 40x_1 + 10x_2 + 6x_3 \leq 55000 + M(1 - y_{iii}) && \leftarrow \text{(iii) silicio massimo} \\
& y_i + y_{ii} + y_{iii} \geq 1 && \leftarrow \text{almeno una proprietà} \\
& 2w \leq y_i + y_{ii} + y_{iii} && \leftarrow \text{bonus solo con almeno due} \\
& x_1, x_2, x_3, x_4 \geq 0, \quad y_i, y_{ii}, y_{iii}, w \in \{0, 1\}
\end{aligned}
$$

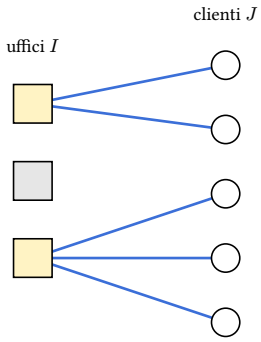
Qui la binaria dice che il vincolo **vale**, quindi il vincolo si spegne con $y = 0$: per questo compare $1 - y$. Se la binaria dicesse «posso violarlo», come nell'esempio dei triangoli, basta sostituire ogni y con $1 - y$: $-My$ al posto di $-M(1 - y)$, e «almeno una proprietà» diventa $y_i + y_{ii} + y_{iii} \leq 2$ (al più due violate). Il bonus funziona come quello dell'home restaurant: con $w = 1$ il vincolo chiede almeno due y a 1, e il $-127w$ nel min spinge w a 1 appena può.

Per i due vincoli con $\geq$ il big-M non serve. Il lato sinistro non è mai negativo, quindi basta moltiplicare il lato destro per la binaria:

$$45x_1 + 50x_2 + 40x_3 + 10000x_4 \geq 45000y_i \qquad 40x_1 + 10x_2 + 6x_3 \geq 32500y_{ii}$$

Con $y = 1$ è il vincolo originale, con $y = 0$ diventa «≥ 0», sempre vero. Sul vincolo con $\leq$ non si può fare: moltiplicato per 0 diventerebbe «≤ 0», che invece obbliga tutto a zero.

Dislocazione ottima di impianti. La società MilanNet vuole aprire fino a n uffici di assistenza per i suoi m clienti, scegliendo fra n siti $I = \{1, \ldots, n\}$. Aprire l'ufficio i costa d_i, e l'ufficio può seguire al massimo u_i clienti; gestire il cliente $j \in J = \{1, \ldots, m\}$ dall'ufficio i costa c_{ij}. Ogni cliente va assegnato a **uno e un solo** ufficio aperto, al costo complessivo (apertura più gestione) minimo.



I quadrati sono i siti (in giallo quelli aperti, $y_i = 1$), i cerchi i clienti. Ogni linea blu è un $x_{ij} = 1$: ogni cliente ne ha esattamente una, e solo verso un ufficio aperto.

Variabili: $y_i \in \{0, 1\}$ vale 1 se apro l'ufficio i; $x_{ij} \in \{0, 1\}$ vale 1 se assegno il cliente j all'ufficio i.

$$
\begin{aligned}
\min z = \sum_{i\in I}\sum_{j\in J} c_{ij}x_{ij} + \sum_{i\in I} d_i y_i & & & \leftarrow \text{gestione + apertura}\\
\sum_{i\in I} x_{ij} = 1 & & j\in J & \leftarrow \text{ogni cliente a un solo ufficio}\\
\sum_{j\in J} x_{ij} \le u_i & & i\in I & \leftarrow \text{al più } u_i \text{ clienti per ufficio}\\
x_{ij} \le y_i & & i\in I, j\in J & \leftarrow \text{solo verso un ufficio aperto}\\
y_i \in \{0,1\}, \quad x_{ij}\in\{0,1\} & & i\in I, j\in J &
\end{aligned}
$$

Il primo vincolo è un semi-assegnamento, il terzo il legame fra le due famiglie di variabili. Gli ultimi due si possono riunire in uno solo, un vincolo per ufficio invece di uno per coppia:

$$\sum_{j\in J} x_{ij} \le u_i y_i \quad i\in I$$

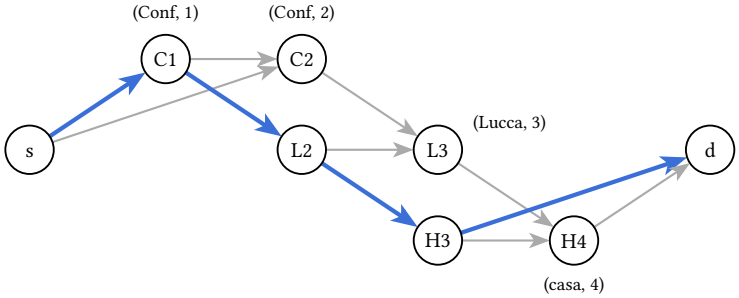
Se apro l'ufficio vale il limite u_i; se non lo apro, a destra c'è 0 e nessun cliente può andarci. Se gli uffici non avessero un limite, si otterrebbe lo stesso effetto con un M grande a piacere, $\sum_{j\in J} x_{ij} \le M y_i$: è il **vincolo di big-M**, lo stesso $x \le My$ dei costi di setup. Serve in generale a legare una variabile non binaria a una binaria.

Un grafo spazio-tempo con le probabilità. Il Prof. Viaggioni è bloccato a una conferenza (Conf) per uno sciopero dei trasporti e deve tornare a casa. Descrive tutte le alternative con un **grafo spazio-tempo** $G = (N, A)$:

- i **nodi** sono coppie (luogo, orario), a partire dalle (Conf, t) per ogni orario t in cui un mezzo dovrebbe partire dalla città della conferenza;
- gli archi **viaggio** $\big((s_i, t_i), (s_j, t_j)\big)$ sono le corse: partenza dal luogo s_i all'orario t_i, arrivo al luogo $s_j \neq s_i$ all'orario $t_j > t_i$;
- gli archi **attesa** restano nello stesso luogo, $s_i = s_j$ e $t_j > t_i$: stare fermi aspettando una coincidenza.

Ogni arco ha un **costo** $c_{ij} \ge 0$ (di solito 0 sulle attese, tranne quando serve un albergo) e una **probabilità** p_{ij} di essere davvero percorribile (< 1 su un'attesa se la coincidenza è stretta o gli alberghi sono pieni). Il professore ha un budget C e vuole che il viaggio riesca con probabilità almeno del 90%, cioè che tutti gli spostamenti scelti vadano a buon fine. Vuole minimizzare il tempo di viaggio, dall'effettiva partenza dalla conferenza all'effettivo arrivo a casa.

Si aggiungono una **super-sorgente** s, collegata a tutti i nodi (Conf, t), e una **super-destinazione** d, a cui sono collegati tutti i nodi (casa, t). Su questi archi artificiali costo 0 e probabilità 1.



Un luogo per riga (C la conferenza, L Lucca, H casa), il numero nel nodo è l'orario: il tempo scorre verso destra. Gli archi orizzontali sono attese, gli altri corse. In blu un cammino: parto dalla conferenza all'orario 1, cambio a Lucca, arrivo a casa all'orario 3.

Variabili: $x_{ij} \in \{0, 1\}$ per ogni arco, vale 1 se il viaggio usa l'arco (i, j) (il mezzo, o l'attesa della coincidenza).

Vincoli. Le x devono descrivere un cammino da s a d: sono i vincoli di conservazione del flusso del cammino minimo (Sezione 1.6), con $b_s = -1$, $b_d = 1$ e 0 negli altri nodi. Il grafo non ha cicli, perché gli archi vanno solo avanti nel tempo. Il budget è come quello dello zaino: $\sum_{(i,j)\in A} c_{ij}x_{ij} \le C$.

Funzione obiettivo: tempo di viaggio = orario di arrivo − orario di partenza. L'arrivo è l'orario t del nodo (casa, t) da cui il cammino entra in d; la partenza quello del nodo (Conf, t) in cui entra da s:

$$\min z = \sum_{(\text{casa},t)\to d} t\, x_{(\text{casa},t),d} - \sum_{s\to(\text{Conf},t)} t\, x_{s,(\text{Conf},t)}$$

Resta la probabilità. Il viaggio riesce se riescono tutti gli archi scelti, quindi la probabilità è il **prodotto**:

$$\prod_{(i,j)\in A : x_{ij}=1} p_{ij} \geq 0.9$$

Non è lineare: è un prodotto, e dipende da quali x valgono 1.

> **Da sapere — logaritmo** (aggiunto, non spiegato in aula)
> Il logaritmo trasforma prodotti in somme, $\log(a \cdot b) = \log a + \log b$, ed è **crescente**: se $a \geq b$ allora $\log a \geq \log b$, quindi applicato ai due lati non gira il verso di una disuguaglianza. Per $0 < p \leq 1$ vale $\log p \leq 0$.

Applico il logaritmo ai due lati, poi la somma sugli archi scelti diventa una somma su tutti gli archi pesata con le x, come nello zaino:

$$\log\left(\prod_{x_{ij}=1} p_{ij}\right) \geq \log 0.9 \quad\to\quad \sum_{x_{ij}=1} \log p_{ij} \geq \log 0.9 \quad\to\quad \sum_{(i,j)\in A} (\log p_{ij}) x_{ij} \geq \log 0.9$$

I $\log p_{ij}$ sono numeri, calcolati sui dati: il vincolo è lineare.

2 Grafi e reti di flusso

Cosa devi saper fare
- portare un problema di flusso alla forma standard con le trasformazioni equivalenti;
- dire se un problema di cammino minimo è vuoto o illimitato;
- calcolare le etichette di un albero, controllare le condizioni di Bellman e dimostrare il teorema;
- eseguire a mano Bellman-Ford (SPT.L) e Dijkstra (SPT.S), e sapere quale usare e quanto costano;
- numerare topologicamente un grafo e trovare i cammini minimi su un grafo aciclico;
- eseguire Kruskal e Prim, e spiegare in cosa Prim è diverso da Dijkstra.

Molti problemi di ottimizzazione hanno la forma di un grafo: a volte è evidente (reti stradali, ferroviarie, di telecomunicazioni, idriche, elettriche), a volte nasce dal modo in cui si scrive il modello. Nel capitolo precedente i problemi su grafi li abbiamo solo **scritti**; qui li **risolviamo**, con algoritmi fatti apposta, a partire dai più semplici.

2.1 Reti di flusso e trasformazioni equivalenti

Una **rete** è un grafo orientato pesato $G = (N, A)$, con $n = |N|$ nodi e $m = |A|$ archi. Sugli archi scorre un flusso, che può essere **discreto** (numero di auto, di messaggi) o **continuo** (petrolio in un oleodotto), e può essere un totale o una quantità per unità di tempo. Il modello è il flusso di costo minimo già visto (Sezione 1.6); qui si fissano i nomi che useranno gli algoritmi.

- **Deficit**: $b_i > 0$ è un nodo di **domanda** (destinazione, pozzo): il bene esce dalla rete. $b_i < 0$ è un nodo di **offerta** (origine, sorgente): il bene entra nella rete. $b_i = 0$ è un nodo **di trasferimento** (il «transito» di prima). Con $D = \{i : b_i > 0\}$ e $O = \{i : b_i < 0\}$, la domanda totale è uguale all'offerta totale: $\sum_{i \in D} b_i = -\sum_{i \in O} b_i$, cioè $\sum_{i \in N} b_i = 0$.
- **Arco** (i, j): un **costo** c_{ij} per unità di flusso, una **capacità inferiore** l_{ij} (il minimo da trasportare) e una **capacità superiore** u_{ij} (il massimo). Spesso $l_{ij} = 0$ e conta solo u_{ij}.
- **Stelle**: $FS(i)$ (stella uscente, forward star) sono gli archi che escono da i, $BS(i)$ (stella entrante, backward star) quelli che entrano.

Un vettore $x = \big(x_{ij}\big)_{(i,j) \in A}$ è un **flusso** se rispetta la conservazione in ogni nodo, ed è un **flusso ammissibile** se rispetta anche le capacità:

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = b_i \quad \forall i \in N \qquad l_{ij} \le x_{ij} \le u_{ij} \quad \forall (i,j) \in A$$

Il suo costo è $c(x) = \sum_{(i,j) \in A} c_{ij} x_{ij}$; minimizzarlo è il problema di **flusso di costo minimo** (MCF). Quasi tutti i problemi del capitolo si riconducono a un MCF.

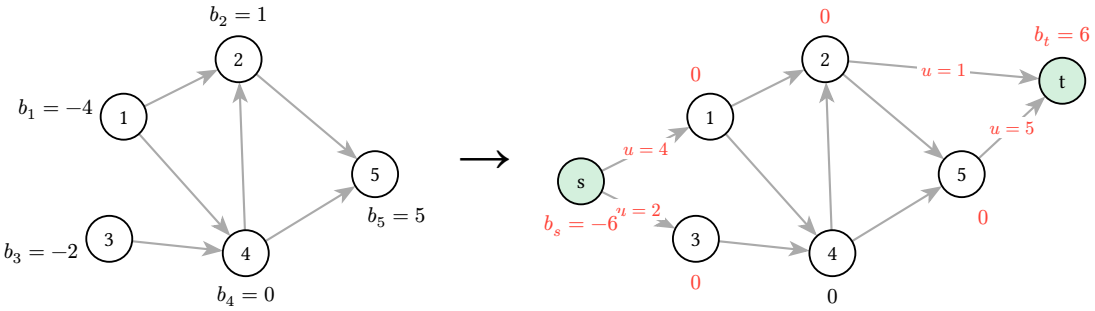
Trasformazioni equivalenti. Gli algoritmi lavorano su una forma semplice del problema. Se l'istanza non la rispetta, se ne costruisce una **equivalente**: un'altra rete che ha le stesse soluzioni.

si assume	se non è così
una sola sorgente e un solo pozzo	sorgenti e pozzi multipli
capacità inferiori nulle	capacità inferiori non nulle
nessuna capacità sui nodi	limiti di capacità sui nodi
offerta = domanda	eccesso di offerta o di domanda

Sorgenti e pozzi multipli. Si costruisce una **rete ampliata** $G' = (N', A')$ con due nodi fittizi, $N' = N \cup \{s, t\}$:

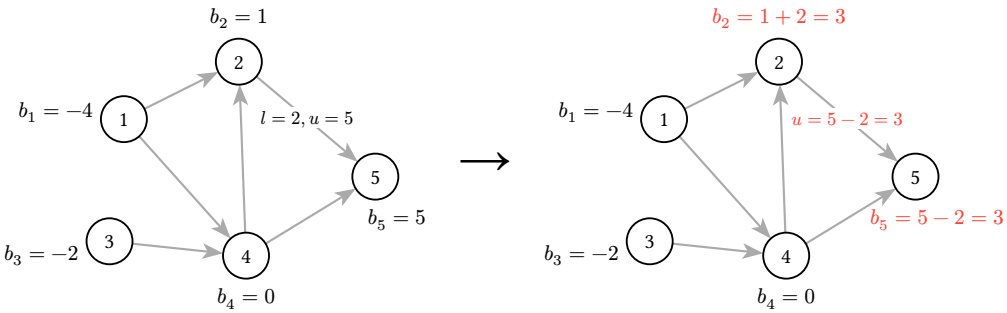
- la **super sorgente** s offre tutto, $b_s = \sum_{j \in O} b_j$, ed è collegata a ogni sorgente $j \in O$ con un arco (s, j) di costo 0 e capacità $u_{sj} = -b_j$;
- il **super pozzo** t chiede tutto, $b_t = \sum_{i \in D} b_i$, e ogni pozzo $i \in D$ è collegato a lui con un arco (i, t) di costo 0 e capacità $u_{it} = b_i$;
- tutti gli altri nodi diventano di trasferimento, $b_i = 0$.

La capacità è il trucco: da s verso j non può passare più di quello che j offriva, quindi ogni vecchia sorgente immette nella rete esattamente quanto prima.



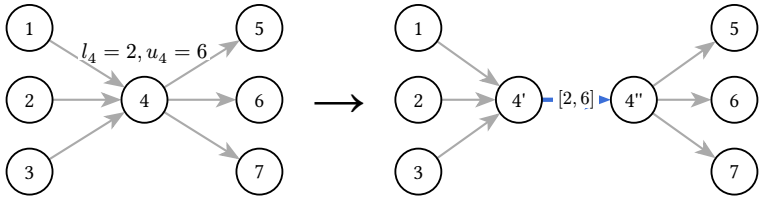
Le sorgenti sono 1 e 3, i pozzi 2 e 5. A destra in rosso quello che cambia: i deficit vecchi vanno a 0, e le capacità degli archi nuovi sono i vecchi deficit.

Capacità inferiori non nulle. Se l'arco (i, j) deve portare almeno $l_{ij} \neq 0$, si immagina che l_{ij} unità ci passino **sempre**, e si toglie dal problema solo la parte che resta da decidere. Il nuovo flusso è $x'_{ij} = x_{ij} - l_{ij}$, e si aggiornano i dati:
- $b'_i = b_i + l_{ij}$: il nodo i ha già spedito l_{ij} unità;
- $b'_j = b_j - l_{ij}$: il nodo j le ha già ricevute;
- $u'_{ij} = u_{ij} - l_{ij}$: sull'arco c'è meno spazio;
- alla funzione obiettivo si aggiunge la costante $c_{ij} l_{ij}$, il costo di quelle unità fisse.

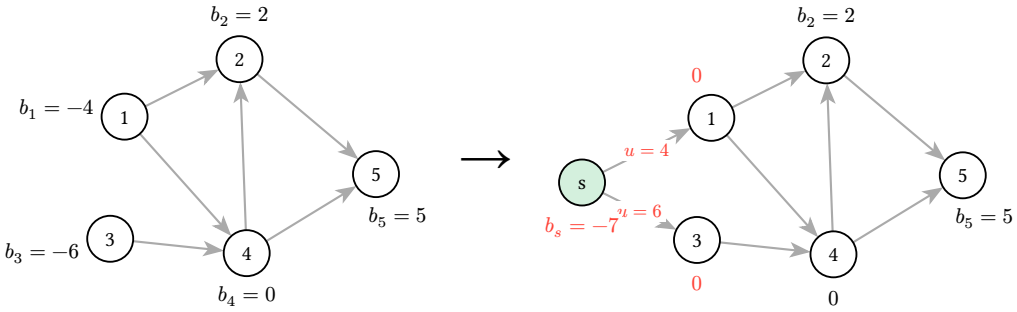


Sull'arco $(2, 5)$, di costo $c_{25} = 1$, passano almeno 2 unità: il 2 deve far arrivare al 5 due unità in più, e il problema nuovo ha un costo fisso di $c_{25} l_{25} = 2$.

Limiti di capacità sui nodi. Un nodo i può avere una capacità massima u_i (non ci passano più di u_i unità) e/o minima l_i. Il modello standard non li sa scrivere, ma li sa scrivere su un arco: si **sdoppia** il nodo in un nodo di ingresso i' e uno di uscita i''. Gli archi che entravano in i entrano in i', quelli che uscivano da i escono da i'', e si aggiunge l'arco fittizio (i', i'') con costo 0 e capacità $[l_i, u_i]$. Tutto quello che passa per il nodo passa per quell'arco.



Eccesso di offerta (o di domanda). A volte quello che una sorgente $i \in O$ offre è un **massimo**, non una quantità da mandare tutta nella rete: l'offerta totale supera la domanda. La conservazione nelle sorgenti diventa una disuguaglianza, $\sum_{BS(i)} x_{ji} - \sum_{FS(i)} x_{ij} \geq b_i$ per $i \in O$, con $b_i < 0$ l'offerta massima. Per tornare a un'uguaglianza si aggiunge una sorgente fittizia s che offre esattamente la domanda, $b_s = -\sum_{j \in D} b_j$; le vecchie sorgenti diventano di trasferimento ($b_i = 0$) e ricevono da s con un arco (s, i) di costo 0 e capacità $u_{si} = -b_i$. Per l'eccesso di domanda, al contrario, si aggiunge un pozzo fittizio.



Le sorgenti potrebbero dare $4 + 6 = 10$, ma i pozzi chiedono solo $2 + 5 = 7$. La sorgente fittizia immette 7, e ogni vecchia sorgente ne riceve al più quanto poteva offrire.

2.2 Cammini minimi: il problema e le condizioni di Bellman

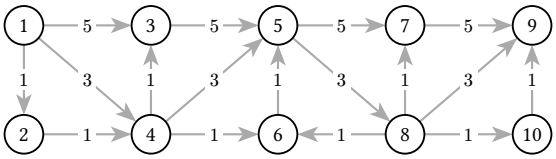
È uno dei problemi più semplici e più importanti sulle reti: lo risolvono i navigatori GPS, Google Maps, le app del trasporto pubblico, su grafi con milioni di nodi e in tempi brevissimi.

Definizione — cammino minimo. Su un grafo orientato $G = (N, A)$, $N = \{1, \ldots, n\}$, ogni arco (i, j) ha un costo $c_{ij} \in \mathbb{R}$ (anche negativo). Un **cammino** P è una sequenza di archi che collega due nodi, e costa $C(P) = \sum_{(i,j) \in P} c_{ij}$. Dati un'origine r e una destinazione t, con $\mathcal{P}_{rt}$ l'insieme dei cammini da r a t, il problema è

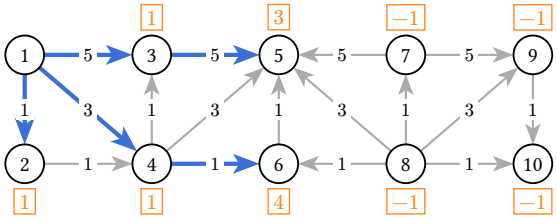
$$\text{(SP)} \quad \min\{C(P) : P \in \mathcal{P}_{rt}\}$$

Come modello è un flusso di costo minimo (Sezione 1.6): $b_r = -1$, $b_t = 1$, $b_i = 0$ negli altri nodi, capacità infinite. Se ci sono più cammini ottimi il flusso si può dividere fra loro, e per avere un cammino solo serve il vincolo di interezza $x \in \{0, 1\}^m$; gli algoritmi che vedremo però restituiscono comunque soluzioni intere.

Prima di cercare il minimo, due cose possono andare storte: il problema può essere **vuoto** o **illimitato**. In tutto il capitolo si usa questo grafo d'esempio (a volte con qualche arco cambiato); i numeri sugli archi sono i costi.



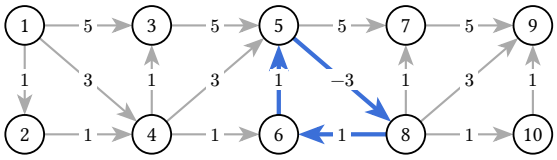
Vuoto: nessun cammino. Se da r non si arriva a t, (SP) non ha soluzioni. Lo si scopre con una **visita** del grafo. Per ogni nodo si tiene l'**etichetta del predecessore** p_j: da quale nodo ci sono arrivato (-1 = mai raggiunto). Si parte con $Q = \{r\}$, la lista dei nodi da visitare; finché Q non è vuota se ne toglie il primo nodo i, e ogni nodo j raggiungibile con un arco (i, j) e mai raggiunto prima prende $p_j = i$ ed entra in fondo a Q.



tolgo	nuovi raggiunti	Q dopo
1	$p_2 = 1, p_3 = 1, p_4 = 1$	$\{2, 3, 4\}$
2	—	$\{3, 4\}$
3	$p_5 = 3$	$\{4, 5\}$
4	$p_6 = 4$	$\{5, 6\}$
5	—	$\{6\}$
6	—	$\varnothing$

Qui alcuni archi sono girati rispetto al grafo d'esempio: in 5 si entra e basta. Le etichette arancioni sono i p_j alla fine, in blu gli archi della visita. I nodi 7, 8, 9 e 10 restano a -1: da 1 non si arriva a 9, e il problema da 1 a 9 è vuoto. Una visita guarda ogni arco una volta sola, quindi costa $O(m)$ (se il grafo è descritto per stelle uscenti).

Illimitato: un ciclo negativo. Può esistere un cammino ma non un cammino **minimo**. Nel grafo d'esempio metto $c_{58} = -3$: il ciclo $5 \to 8 \to 6 \to 5$ costa $-3 + 1 + 1 = -1$.



Un cammino da 1 a 9 può passare da 5, fare il giro del ciclo e tornare in 5 avendo speso 1 in meno. Ogni giro in più toglie un'altra unità: il costo scende senza fine, e il problema è **inferiormente illimitato** (il minimo è $-\infty$).

> **Illimitatezza.** Un problema di cammino minimo è inferiormente illimitato $\iff$ esiste un ciclo orientato di costo negativo **raggiungibile dall'origine** e **dal quale si può raggiungere la destinazione**.

Non basta che il ciclo esista: se il cammino non lo può toccare, «lo guarda da lontano» e la soluzione c'è lo stesso. Se esiste un ciclo negativo lo si può scoprire in tempo polinomiale (vedremo come).

Un cammino che passa due volte per lo stesso nodo contiene un ciclo: si dice **non semplice**. Senza cicli negativi c'è sempre una soluzione ottima **semplice**, perché togliere un ciclo non negativo non peggiora il costo. Con i cicli negativi, invece, cercare il cammino **semplice** di costo minimo è **NP-arduo**: almeno difficile quanto ogni problema in NP, e nessun algoritmo noto lo risolve in tempo polinomiale. Non è un problema «semplice», e qui non si affronta.

Una condizione **sufficiente ma non necessaria** per non avere cicli negativi è che tutti i costi siano ≥ 0. Si possono avere costi negativi senza cicli negativi: con $c_{12} = -1$ nel grafo d'esempio non cambia niente, perché in 1 non entra nessun arco e quindi l'arco $(1, 2)$ non sta in nessun ciclo.

Dal cammino all'albero. Se il cammino minimo da 1 a 6 è $1 \to 2 \to 4 \to 6$, allora $1 \to 2 \to 4$ è il cammino minimo da 1 a 4, e $1 \to 2$ quello da 1 a 2: **ogni pezzo di un cammino minimo è minimo**. Se ci fosse un cammino migliore da 1 a 4, sostituendolo si otterrebbe un cammino migliore anche da 1 a 6. Quindi calcolare un cammino minimo vuol dire calcolarne tanti insieme, e conviene cercarli **tutti**: i cammini minimi dalla radice r a ogni altro nodo.

> **Definizione — albero dei cammini minimi (SPT).** Data una radice r, il problema è trovare un cammino minimo P_i da r a ogni nodo $i \neq r$, cioè $\min\left\{\sum_{i \neq r} C(P_i) : P_i \in \mathcal{P}_{ri}\right\}$. L'unione dei P_i si può sempre prendere come un **albero radicato in r** (Shortest Path Tree, SPT): ogni arco è orientato da r verso le foglie, entra in ogni nodo $i \neq r$ un solo arco, e gli archi sono $n - 1$. Il cammino dell'albero da r a un nodo intermedio è a sua volta minimo.

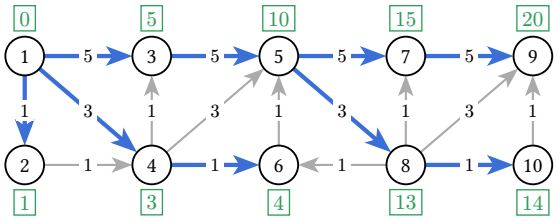
Per descrivere tutti i cammini bastano quindi $n - 1$ informazioni: per ogni nodo, il suo **predecessore** p_i nell'albero. Anche l'SPT è un flusso di costo minimo: la radice è una sorgente che manda un'unità a

ciascun altro nodo, $b_r = -(n-1)$, e ogni nodo $i \neq r$ è un pozzo con $b_i = 1$; x_{ij} conta quanti cammini usano l'arco. Senza cicli negativi questo problema ha un ottimo finito.

Un nodo i non raggiungibile da r farebbe fallire tutto. Si aggiunge allora un arco fittizio (r, i) con costo $M = (n-1)c_{\max} + 1$, dove $c_{\max}$ è il costo più alto: più di qualunque cammino vero (un cammino semplice ha al più $n-1$ archi). Il cammino minimo verso i usa quell'arco **solo se** non esistono cammini reali.

Verificare un albero: le etichette. Una visita costruisce un albero radicato in r che raggiunge tutti i nodi. Basta tenere conto, mentre si visita, di quanto costa arrivare a ogni nodo:

Definizione — etichette di un albero. Dato un albero $T = (N, A_T)$ radicato in r, d_i è il costo del cammino da r a i **nell'albero**, $d_i = C(P_i^T)$, con $d_r = 0$. Su ogni arco (i, j) dell'albero vale $d_i + c_{ij} = d_j$.

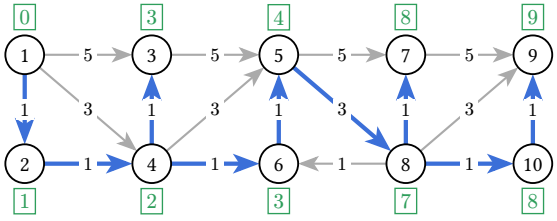


L'albero della visita sul grafo d'esempio (in blu), con le etichette d nei riquadri verdi: per arrivare in 7 passo da 3 e 5, e pago $5 + 5 + 5 = 15$.

È l'albero dei cammini minimi? No: arrivare in 4 passando da 2 costa $d_2 + c_{24} = 1 + 1 = 2$, meno di $d_4 = 3$.

In generale, per ogni arco (i, j) **fuori** dall'albero si controlla se $d_i + c_{ij} < d_j$: se sì, passando da i si arriva in j spendendo meno, e l'albero non è ottimo. Si sostituisce l'arco che entrava in j con (i, j), e l'etichetta di j cala, **a cascata** insieme a quelle di tutti i nodi che nell'albero stanno sotto j. Ecco un giro sugli archi fuori dall'albero, in ordine:

arco (i, j)	$d_i + c_{ij}$	d_j	migliora?	etichette che cambiano
$(2, 4)$	2	3	✓	$d_4 = 2$, $d_6 = 3$
$(4, 3)$	3	5	✓	$d_3 = 3$, $d_5 = 8$, $d_7 = 13$, $d_8 = 11$, $d_9 = 18$, $d_{10} = 12$
$(4, 5)$	5	8	✓	$d_5 = 5$, $d_7 = 10$, $d_8 = 8$, $d_9 = 15$, $d_{10} = 9$
$(6, 5)$	4	5	✓	$d_5 = 4$, $d_7 = 9$, $d_8 = 7$, $d_9 = 14$, $d_{10} = 8$
$(8, 6)$	8	3	✗	—
$(8, 7)$	8	9	✓	$d_7 = 8$, $d_9 = 13$
$(8, 9)$	10	13	✓	$d_9 = 10$
$(10, 9)$	9	10	✓	$d_9 = 9$



L'albero alla fine. Ora nessun arco del grafo ha $d_i + c_{ij} < d_j$: non si può migliorare niente, e il teorema che segue dice che è l'albero dei cammini minimi.

Definizione — condizioni di Bellman. Un vettore di etichette d verifica le condizioni di Bellman se

$$d_i + c_{ij} \geq d_j \quad \forall (i, j) \in A, \qquad d_r = 0$$

Sugli archi dell'albero vale con l'uguale, sugli altri dice che passare da quell'arco non conviene.

Teorema di Bellman. Un albero $T = (N, A_T)$ radicato in r è un albero dei cammini minimi $\iff$ il suo vettore di etichette d verifica le condizioni di Bellman.

Dimostrazione di $\Longrightarrow$, per assurdo. Sia T un albero dei cammini minimi, e supponiamo che una condizione sia violata: per un arco (i, j) vale $d_i + c_{ij} < d_j$.
1. d_i è il costo di un cammino da r a i; aggiungendo l'arco (i, j) ottengo un cammino da r a j che costa $d_i + c_{ij}$.
2. Questo costa meno di d_j, il cammino dell'albero verso j: quindi quello non è minimo.
3. Allora T non è un albero dei cammini minimi: assurdo.

Per l'altra direzione serve un lemma.

Lemma di Bellman. Se d verifica le condizioni di Bellman, allora per ogni nodo $i \neq r$ e per **ogni** cammino P da r a i nel grafo (anche non semplice): $d_i \leq C(P)$.

Dimostrazione. Sia P fatto degli archi $(i_1, i_2), (i_2, i_3), \ldots, (i_{k-1}, i_k)$, con $i_1 = r$ e $i_k = i$.
1. Scrivo la condizione di Bellman per ogni arco di P: $d_{i_1} + c_{i_1 i_2} \geq d_{i_2}$, $d_{i_2} + c_{i_2 i_3} \geq d_{i_3}$, $\ldots$, $d_{i_{k-1}} + c_{i_{k-1} i_k} \geq d_{i_k}$.
2. Sommo tutte le disuguaglianze (sommando, il verso resta). Ogni d intermedio compare a sinistra e a destra e si cancella: resta $d_{i_1} + C(P) \geq d_{i_k}$.
3. Con $d_{i_1} = d_r = 0$ e $i_k = i$: $C(P) \geq d_i$.

Dimostrazione di $\Longleftarrow$. Se d verifica le condizioni di Bellman, per il lemma nessun cammino verso i costa meno di d_i. Ma d_i è il costo del cammino dell'albero verso i: quel cammino è minimo, per ogni i. Quindi T è un albero dei cammini minimi.

Il costo dell'albero dei cammini minimi è la somma dei costi dei cammini, cioè la **somma delle etichette** $\sum_i d_i$, non la somma dei costi dei suoi archi: un arco vicino alla radice sta in molti cammini e va contato ogni volta. Nell'esempio la somma delle etichette fa 45, quella degli archi dell'albero solo 11. Il prof avverte: è una domanda tipica dei compiti, e spesso viene sbagliata.

2.3 Algoritmi per l'albero dei cammini minimi: Bellman-Ford e Dijkstra

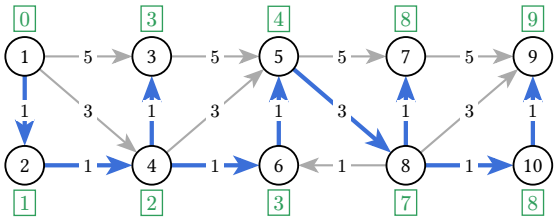
Visitare il grafo e poi correggere l'albero arco per arco funziona, ma costringe a molte visite. Conviene correggere le etichette **mentre** si visita: è l'**algoritmo SPT**. Riceve il grafo G, i costi c e la radice r, e restituisce le etichette dei predecessori p e dei costi d.

```
procedure (p, d) = SPT(G, c, r)
  foreach (i in N) do { p[i] = r; d[i] = M; }   // inizializzazione
  d[r] = 0;  Q = { r };
  do {
    select i from Q;  Q = Q \ { i };
    foreach ((i, j) in FS(i)) do
      if (d[i] + c[i][j] < d[j]) then {          // (i, j) viola Bellman
        d[j] = d[i] + c[i][j];  p[j] = i;
        Q = Q ∪ { j };
      }
  } while (Q ≠ ∅);
```

riga	cosa fa
`p[i] = r; d[i] = M`	ogni nodo «arriva dalla radice» con costo M grande: è come aggiungere gli archi fittizi (r, i) di costo M. Se alla fine un nodo ha ancora $d_i = M$, non era raggiungibile
`d[r] = 0; Q = {r}`	arrivare in r costa 0 (come un arco (r, r) di costo 0); in Q, la lista dei nodi da esaminare, c'è solo r
`select i from Q`	tolgo un nodo da Q; **quale** nodo scelgo è ciò che distingue gli algoritmi
`foreach (i, j) in FS(i)`	guardo tutti gli archi che escono da i
`if d[i] + c[i][j] < d[j]`	l'arco viola la condizione di Bellman: passando da i arrivo in j spendendo meno
`d[j] = …; p[j] = i`	aggiorno costo e predecessore di j: ora ci arrivo da i
`Q = Q ∪ {j}`	l'etichetta di j è cambiata, quindi i suoi archi uscenti vanno ricontrollati: j entra in Q (se c'è già, resta dov'è)
`while Q ≠ ∅`	quando Q è vuota nessun arco viola più Bellman: per il teorema l'albero è ottimo

SPT.L.Queue: l'algoritmo di Bellman-Ford. La scelta più semplice: Q è una **fila** (FIFO, first in first out), si estrae sempre il primo e i nuovi entrano in fondo. L'algoritmo si chiama SPT.L.Queue (L perché Q è una lista, Queue perché è una fila) o **algoritmo di Bellman-Ford**. Sul grafo d'esempio con radice 1, guardando gli archi uscenti in ordine di nodo d'arrivo:

passo	estraggo i	archi (i, j) che violano Bellman → nuova d_j	Q dopo
1	1	$d_2 = 1$, $d_3 = 5$, $d_4 = 3$	$\{2, 3, 4\}$
2	2	$d_4 = 2$	$\{3, 4\}$
3	3	$d_5 = 10$	$\{4, 5\}$
4	4	$d_3 = 3$, $d_5 = 5$, $d_6 = 3$	$\{5, 3, 6\}$
5	5	$d_7 = 10$, $d_8 = 8$	$\{3, 6, 7, 8\}$
6	3	—	$\{6, 7, 8\}$
7	6	$d_5 = 4$	$\{7, 8, 5\}$
8	7	$d_9 = 15$	$\{8, 5, 9\}$
9	8	$d_7 = 9$, $d_9 = 11$, $d_{10} = 9$	$\{5, 9, 7, 10\}$
10	5	$d_8 = 7$	$\{9, 7, 10, 8\}$
11	9	—	$\{7, 10, 8\}$
12	7	—	$\{10, 8\}$
13	10	$d_9 = 10$	$\{8, 9\}$
14	8	$d_7 = 8$, $d_{10} = 8$	$\{9, 7, 10\}$
15	9	—	$\{7, 10\}$
16	7	—	$\{10\}$
17	10	$d_9 = 9$	$\{9\}$
18	9	—	$\varnothing$



Il risultato: in blu l'albero dei cammini minimi (l'arco blu che entra in j viene da p_j), nei riquadri le etichette d. Gli archi grigi rispettano tutti Bellman.

Lo stesso nodo può **rientrare** in Q più volte: 3 rientra quando da 4 si scopre una strada migliore, e ogni volta che un'etichetta cala devono essere ricontrollati anche i nodi che vengono dopo. L'algoritmo termina sempre?

> **Cicli negativi e Bellman.** Se esiste un ciclo orientato di costo negativo, **nessun** vettore d verifica le condizioni di Bellman.

Dimostrazione, per assurdo. Sia $L = (i_1, i_2, \ldots, i_k, i_1)$ un ciclo con $C(L) < 0$, e supponiamo che d verifichi Bellman.

1. Scrivo la condizione per ogni arco del ciclo: $d_{i_1} + c_{i_1 i_2} \geq d_{i_2}, \quad \ldots, \quad d_{i_k} + c_{i_k i_1} \geq d_{i_1}$.
2. Sommo: questa volta **tutte** le d compaiono una volta per lato, perché il ciclo torna al nodo di partenza, e si cancellano. Resta $0 \leq C(L)$.
3. Ma $C(L) < 0$: contraddizione. Con un ciclo negativo il problema non ha soluzione finita.

Quindi, per l'algoritmo:

- se c'è un ciclo negativo **raggiungibile da r**, l'algoritmo **non termina**: le etichette del ciclo continuano a scendere. Sul grafo con $c_{58} = -3$, l'etichetta di 8 nelle prime 40 estrazioni vale via via 2, 1, 0, -1, -2, -3, ...;
- se c'è un ciclo negativo **non raggiungibile da r**, l'algoritmo termina e dà l'albero dei cammini minimi dei soli nodi raggiungibili;
- se non ci sono cicli negativi, l'algoritmo termina.

L'ultimo punto si vede così:

1. in ogni momento ogni etichetta $d_i < M$ è il costo di un cammino vero da r a i (non per forza il minimo);
2. a ogni passo le etichette non aumentano mai: o calano o restano uguali;
3. senza cicli negativi un'etichetta non può calare all'infinito, perché i cammini da cui viene, tolti i cicli, sono finiti;
4. quindi dopo un numero finito di passi le etichette arrivano al costo dei cammini minimi e Q si svuota.

Con Q gestita come fila si può dire di più:

> **Proprietà della fila.** Se non esistono cicli orientati di costo negativo e Q è una fila (FIFO), nessun nodo entra in Q più di $n - 1$ volte.

La fila visita il grafo «a ventaglio»: prima i nodi a un arco dalla radice, poi quelli a due archi, e così via. Quando un nodo rientra in Q è perché ha trovato un cammino che costa meno ma ha **più archi**. Un cammino semplice ha al più $n - 1$ archi, quindi gli aggiornamenti di un nodo sono al più $n - 1$.

Da qui due conseguenze:

- **complessità $O(nm)$**: ogni nodo esce da Q al più $n - 1$ volte, e ogni volta guarda i suoi archi uscenti; su tutti i nodi sono al più m archi per giro. È polinomiale, ma su un grafo **denso** ($m \approx n^2$) diventa $O(n^3)$: poco efficiente;
- **si scoprono i cicli negativi**: si conta quante volte ogni nodo entra in Q. Se un nodo ci entra n volte, c'è un ciclo negativo raggiungibile: ci si ferma e si dichiara il problema illimitato.

Altre code (il prof le ha date solo come idea, non le chiede). Con la fila si formano dei «treni» di aggiornamenti: quando un'etichetta cala, tutti i nodi dopo di lui rientrano in coda. Altre regole provano a evitarlo:

- **Dequeue** (coda a due uscite): si estrae dalla testa; un nodo mai visto ($d = M$) entra in fondo, uno già visto ($d < M$) entra in **testa**, perché i nodi dopo di lui vanno aggiornati subito. Complessità teorica $O(n2^n)$, ma in pratica è più veloce. Perde il controllo dei cicli negativi, che dipendeva dalla fila: si usa quando si sa che non ce ne sono;
- **LIFO**: l'ultimo entrato è il primo a uscire, $O(n2^n)$;
- **2-queue**: due code, una prioritaria per i nodi già visti ($d < M$) e una per quelli nuovi; si svuota prima la prioritaria. $O(nm^2)$;
- **Threshold**: una soglia T; nella coda prioritaria i nodi con $d_i \leq T$, nell'altra quelli con $d_i > T$. Con un buon T è molto efficiente.

SPT.S: l'algoritmo di Dijkstra. Perché non scegliere direttamente in base alle etichette? Si estrae da Q il nodo con l'**etichetta più piccola** (a parità, quello col nome più piccolo). È l'**algoritmo di Dijkstra**, o SPT.S (S per Shortest-first). Stesso grafo, stessa radice:

passo	estraggo i	archi (i,j) che violano Bellman $\rightarrow$ nuova d_j	Q dopo
1	1	$d_2 = 1$, $d_3 = 5$, $d_4 = 3$	$\{2, 3, 4\}$
2	2	$d_4 = 2$	$\{3, 4\}$
3	4	$d_3 = 3$, $d_5 = 5$, $d_6 = 3$	$\{3, 5, 6\}$
4	3	—	$\{5, 6\}$
5	6	$d_5 = 4$	$\{5\}$
6	5	$d_7 = 9$, $d_8 = 7$	$\{7, 8\}$
7	8	$d_7 = 8$, $d_9 = 10$, $d_{10} = 8$	$\{7, 9, 10\}$
8	7	—	$\{9, 10\}$
9	10	$d_9 = 9$	$\{9\}$
10	9	—	$\varnothing$

Stesse etichette di Bellman-Ford, identiche, ma in 10 estrazioni invece di 18: **ogni nodo esce da Q una volta sola**.

Teorema di Dijkstra. Se tutti i costi sono non negativi, ogni nodo viene inserito ed estratto da Q **al più una volta**. («Al più» perché un nodo non raggiungibile non entra mai.)

Serve un lemma: **con costi non negativi le etichette dei nodi estratti, nell'ordine in cui escono, non decrescono**. Se $d^k_{i_k}$ è l'etichetta del nodo i_k estratto al passo k, allora $d^1_{i_1} \leq d^2_{i_2} \leq \ldots \leq d^n_{i_n}$. Infatti si estrae sempre la più piccola, e i nodi aggiornati dopo ricevono $d_i + c_{ij} \geq d_i$.

Dimostrazione, per assurdo. Supponiamo che un nodo i esca due volte: al passo k con etichetta d^k_i e al passo $h > k$ con d^h_i.

1. Per rientrare in Q, la sua etichetta deve essere calata: $d^h_i < d^k_i$.
2. Ma per il lemma le etichette estratte non decrescono: $d^k_i \leq d^h_i$.
3. Le due cose non possono valere insieme: assurdo.

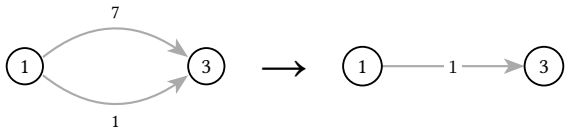
Quindi, con costi ≥ 0, **quando un nodo esce da Q la sua etichetta è definitiva**: è già il costo del cammino minimo.

Complessità di Dijkstra, con costi $c_{ij} \geq 0$:

- ogni nodo è estratto una volta sola: n estrazioni;

- ogni arco è guardato una volta sola, quando esce il suo nodo di partenza: m in tutto;
- ogni estrazione cerca il minimo in Q, $i = \text{argmin}\{d_j : j \in Q\}$, e senza altri accorgimenti costa $O(n)$, perché Q ha al più n nodi.

In totale $O(n^2 + m) = O(n^2)$, perché $m \leq n^2$. Questo vale se non ci sono **archi paralleli** (due archi con stessi estremi e stesso verso); se ci sono, basta tenere quello che costa meno.



Bellman-Ford costa $O(nm)$: su un grafo denso ($m \approx n^2$) fa $O(n^3)$, e Dijkstra con $O(n^2)$ è meglio. Su un grafo **sparso** ($m \approx n$) Dijkstra migliora ancora tenendo Q in un **heap binario**, una struttura che dà il minimo in $O(\log n)$. Le n estrazioni e gli m aggiornamenti costano $O(\log n)$ ciascuno: $O(n \log n + m \log n) = O(m \log n)$, meglio di $O(n^2)$ sui grafi sparsi.

	complessità	**quando**
Dijkstra (SPT.S), min per scansione	$O(n^2)$	costi ≥ 0, grafo denso
Dijkstra (SPT.S) con heap	$O(m \log n)$	costi ≥ 0, grafo sparso
Bellman-Ford (SPT.L, fila)	$O(nm)$	ci sono costi negativi: scopre anche i cicli negativi

Dijkstra con costi negativi: il teorema non vale più, i nodi possono rientrare in Q e la complessità può diventare esponenziale; con un ciclo negativo Dijkstra non termina. Siccome guardando il grafo non si sa se un ciclo negativo c'è, con costi negativi si usa Bellman-Ford.

Poiché in Dijkstra l'etichetta di un nodo è definitiva appena il nodo esce da Q, l'algoritmo serve anche quando interessa il cammino minimo verso **un solo** nodo t: ci si ferma appena t esce, senza esplorare il resto del grafo. Allo stesso modo, per un **sottoinsieme di destinazioni** $S \subseteq N \setminus \{r\}$ ci si ferma quando sono usciti tutti i nodi di S.

Radici multiple. Se si vuole il cammino minimo verso t partendo da **una qualunque** delle sorgenti $R = \{r_1, \ldots, r_k\}$, si aggiunge una radice fittizia r_0 collegata a ogni r_i con un arco di costo 0, $c_{r_0 r_i} = 0$, e si calcolano i cammini minimi da r_0, fermandosi quando esce t. Più semplicemente, si parte con tutte le r_i in Q ed etichetta 0.

2.4 Cammini minimi su grafi aciclici

Dijkstra è il più efficiente, ma solo senza costi negativi, perché i costi negativi possono creare cicli negativi. Se il grafo **non ha cicli**, il problema non si pone, e si può fare ancora meglio. Come si capisce se un grafo è aciclico?

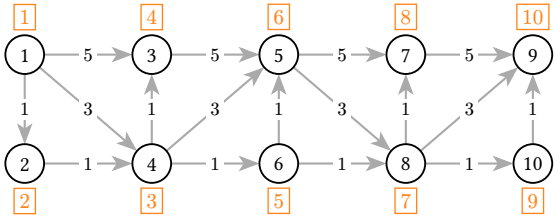
Numerazione topologica. Un grafo orientato è aciclico $\iff$ esiste una **numerazione topologica** dei nodi: si possono numerare $1, 2, \ldots, n$ in modo che per ogni arco $(i, j) \in A$ valga $i < j$.

Con un ciclo nessuna numerazione funziona: seguendo il ciclo i numeri dovrebbero crescere sempre, e poi tornare al primo. La numerazione si costruisce a passi:

1. si prende un nodo che **non ha archi entranti** e gli si dà il primo numero non ancora usato (se ce ne sono più d'uno, qui quello col nome più piccolo);
2. si cancellano i suoi archi uscenti, e si torna al passo 1.

Se a un certo punto nessun nodo rimasto è privo di archi entranti, il grafo ha un ciclo. Il prof avverte che una domanda frequente è «questa numerazione è topologica? sì o no».

Esempio: il grafo d'esempio con l'arco $(6, 8)$ al posto di $(8, 6)$. Nel grafo originale, invece, il ciclo $5 \to 8 \to 6 \to 5$ blocca la numerazione dopo 4 nodi.



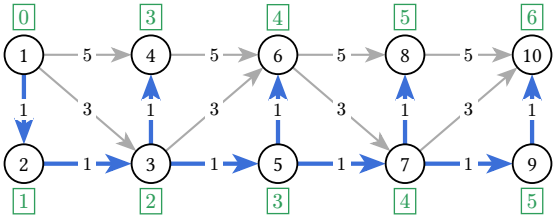
Nei riquadri arancioni il numero topologico di ogni nodo. L'ordine in cui i nodi vengono numerati: 1,2,4,3,6,5,8,7, 10,9.

In 1 non entra niente: prende il numero 1. Cancellati i suoi archi, l'unico nodo senza archi entranti è 2, poi 4; a quel punto sono liberi sia 3 sia 6, e si prende 3.

Una volta numerati i nodi, l'ordine in cui esaminarli è già deciso: **SPT.Acyclic** prende i nodi in ordine $1, 2, \ldots, n$ e per ognuno controlla gli archi uscenti come nell'algoritmo SPT. Non serve la coda: quando si arriva al nodo i, tutti gli archi che entrano in i vengono da nodi con numero più piccolo, già esaminati, quindi d_i è già definitiva. Ogni arco è guardato una volta: $O(m)$, e anche la numerazione costa $O(m)$.

Qui sotto lo stesso grafo con i nodi rinominati col loro numero topologico (ora ogni arco va da un numero più piccolo a uno più grande), radice 1:

esamino i	**archi (i, j) che migliorano → nuova d_j**
1	$d_2 = 1, d_3 = 3, d_4 = 5$
2	$d_3 = 2$
3	$d_4 = 3, d_5 = 3, d_6 = 5$
4	—
5	$d_6 = 4, d_7 = 4$
6	$d_8 = 9$
7	$d_8 = 5, d_9 = 5, d_{10} = 7$
8	—
9	$d_{10} = 6$
10	—



Senza cicli si possono fare altre cose:

- **costi negativi**: su un grafo aciclico non ci sono cicli negativi, quindi SPT.Acyclic funziona anche con costi negativi, sempre in $O(m)$;
- **cammini di costo massimo**: si moltiplicano per -1 i costi di tutti gli archi e si cercano i cammini minimi;
- **altre radici**: per l'albero da un'altra radice basta considerare i nodi dal numero della radice in avanti (gli altri non si raggiungono).

2.5 Albero di copertura di costo minimo

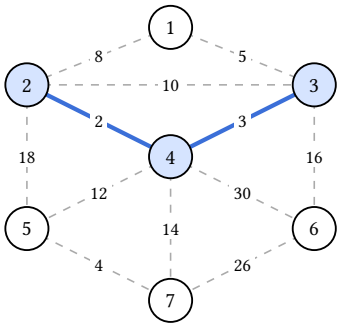
Il problema l'abbiamo già modellato (Sezione 1.3), e il prof avverte che la formulazione la può chiedere. Dato un grafo **non orientato** $G = (V, E)$ con costi c_{ij} sui lati, si cerca un **albero di copertura** $T = (V, E_T)$, cioè un sottografo con $n - 1$ lati, connesso e senza cicli, di costo minimo $c(T) = \sum_{\{i,j\} \in E_T} c_{ij}$.

È uno dei pochi problemi di ottimizzazione che si risolvono all'ottimo con un algoritmo **greedy** («goloso»: a ogni passo fa la scelta che sembra migliore in quel momento, e non torna indietro). Somiglia al problema dell'SPT, anche se è su grafi non orientati, e serve per le reti elettriche e di telecomunicazioni.

Si assume che G sia **connesso** e che tutti i costi siano **positivi**. Se G non è connesso l'albero di copertura non esiste, e si parla di **foresta** di copertura di costo minimo: si aggiungono lati fittizi fra nodi che non si raggiungono, con un costo molto grande, per esempio $M = c_{\max} + 1$, e l'algoritmo li userà solo se necessario. Se ci sono costi negativi si aggiunge a tutti i lati la stessa costante $\delta = -c_{\min} + 1$ ($c_{\min}$ il costo più piccolo): tutti i costi diventano positivi e la soluzione ottima non cambia, perché ogni albero ha $n - 1$ lati e il suo costo cresce di $(n-1)\delta$, uguale per tutti.

Lo schema generale: Greedy-MST. L'albero si costruisce passo passo, decidendo lato per lato se includerlo o escluderlo. Si tengono due insiemi di lati, sempre disgiunti ($S \subseteq E$, $R \subseteq E$, $S \cap R = \varnothing$):
- S: i lati **scelti**, già nell'albero;
- R: i lati **scartati**, che nell'albero non entreranno mai.

Le operazioni sono due. Servono i **tagli**: presi due insiemi di nodi non vuoti e disgiunti V' e V'' con $V' \cup V'' = V$, il taglio $E(V', V'')$ è l'insieme dei lati con un estremo in V' e l'altro in V'', i «ponti» fra le due parti.



- **Inserzione**: si prende un taglio in cui nessun lato è già in S, e fra i suoi lati non scartati si sceglie quello di **costo minimo**, che entra in S. Garantisce la connessione: per passare da una parte all'altra serve un ponte, e si prende il più economico.
- **Cancellazione**: si prende un ciclo C senza lati scartati, e fra i suoi lati non scelti si scarta quello di **costo massimo**, che va in R. Evita i cicli: un ciclo nell'albero non ci può stare, e il lato più caro non serve.

Nel disegno $S = \{\{2,4\},\{3,4\}\}$ (in blu). Col taglio fra i nodi azzurri e gli altri, il lato più economico è $\{1,3\}$ (5).

> **Lemma di correttezza.** Siano S i lati scelti e R quelli scartati, e supponiamo che esista un albero di copertura ottimo T^* che contiene tutti i lati di S e nessuno di R. Allora, dopo un'inserzione o una cancellazione, anche i nuovi S' e R' hanno questa proprietà.

Partendo da $S = R = \varnothing$ la proprietà vale, e ogni operazione la mantiene: per induzione Greedy-MST finisce sempre con un albero di copertura di costo minimo (se esiste), **in qualunque ordine** si facciano le operazioni.

```
procedure S = Greedy-MST(G, c)
  S = ∅;  R = ∅;
  do
    applica Inserzione o Cancellazione;
  while (S ∪ R ⊊ E and |S| < n - 1);
```

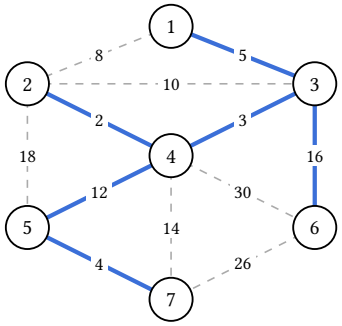
Si va avanti finché non sono stati classificati tutti i lati ($S \cup R = E$) o finché S ha $n - 1$ lati: a quel punto è un albero completo, e un lato in più creerebbe un ciclo. La complessità è $O(m)$, perché si valuta al più ogni lato. Gli algoritmi pratici sono casi particolari dello schema, che si distinguono per l'ordine delle operazioni: **Kruskal** sceglie sempre il lato più economico che non crea cicli, **Prim** fa crescere un albero aggiungendo nodi vicini. Entrambi rispettano il lemma, quindi sono ottimi.

Algoritmo di Kruskal. Si ordinano tutti i lati per costo crescente, $c_1 \le c_2 \le \ldots \le c_m$, e si esaminano uno alla volta in quest'ordine:

- se il lato **non forma un ciclo** con quelli già in S, entra in S: è un'inserzione, perché unisce due pezzi separati e, fra i lati rimasti, è il più economico del taglio fra uno dei due pezzi e il resto;
- se **forma un ciclo**, va in R: è una cancellazione, perché è il più caro del ciclo (gli altri lati del ciclo sono stati esaminati prima).

Ci si ferma appena $|S| = n - 1$, anche se restano lati da esaminare.

lato	costo	esito
$\{2,4\}$	2	entra in S
$\{3,4\}$	3	entra in S
$\{5,7\}$	4	entra in S
$\{1,3\}$	5	entra in S
$\{1,2\}$	8	ciclo: va in R
$\{2,3\}$	10	ciclo: va in R
$\{4,5\}$	12	entra in S
$\{4,7\}$	14	ciclo: va in R
$\{3,6\}$	16	entra in S



Al lato $\{3,6\}$ S ha già $n - 1 = 6$ lati: ci si ferma, e $\{2,5\}$, $\{4,6\}$, $\{6,7\}$ non si guardano nemmeno. L'albero (in blu) costa 2 + 3 + 4 + 5 + 12 + 16 = 42.

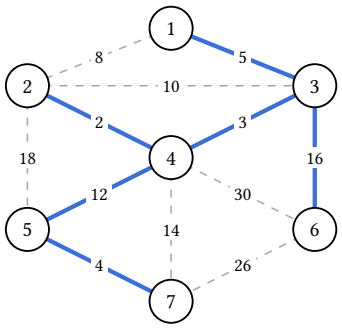
Complessità di Kruskal: una fase di ordinamento, $O(m \log m)$, e una di selezione, al più m lati. Per controllare in fretta se un lato chiude un ciclo si usa una struttura dati **Union-Find**, che lo fa a costo quasi costante: l'ordinamento domina. In un grafo connesso $n - 1 \leq m \leq \frac{n(n-1)}{2} = O(n^2)$, quindi $\log m \leq \log n^2 = 2 \log n$, e la complessità si scrive anche $O(m \log m) = O(m \log n)$.

Algoritmo di Prim. Somiglia agli algoritmi dei cammini minimi: si sceglie un nodo qualsiasi come radice r e si fa crescere un albero da lì, con un'etichetta d_v per nodo e il predecessore p_v. Qui però d_v è il costo del **lato** più economico che collega v all'albero costruito fin qui.

1. **Inizializzazione**: per ogni nodo $p_v = r$ e $d_v = M$ (come degli archi fittizi di costo infinito); per la radice $d_r = -M$; $Q = \{r\}$.
2. Si estrae da Q il nodo u con etichetta **minima** e gli si mette $d_u = -M$: ora è nell'albero, con il lato $\{p_u, u\}$.
3. Per ogni lato $\{u, v\}$: se $c_{uv} < d_v$, allora $d_v = c_{uv}$ e $p_v = u$, e v entra in Q se non c'è già.
4. Si torna al passo 2 finché Q non è vuota.

L'etichetta $-M$ dei nodi già nell'albero non può mai essere migliorata, quindi quei nodi non vengono più toccati e non si formano cicli: Prim non fa mai cancellazioni. Ogni estrazione, tranne la prima, è un'**inserzione** sul taglio fra i nodi con etichetta $-M$ (l'albero) e tutti gli altri. Radice 1:

estraggo u	entra	etichette aggiornate	costo
1	–	$d_2 = 8, d_3 = 5$	0
3	$\{1,3\}$	$d_4 = 3, d_6 = 16$	5
4	$\{3,4\}$	$d_2 = 2, d_5 = 12, d_7 = 14$	8
2	$\{4,2\}$	–	10
5	$\{4,5\}$	$d_7 = 4$	22
7	$\{5,7\}$	–	26
6	$\{3,6\}$	–	42



Stesso albero di Kruskal, di costo 42. Per esempio, quando esce 4 il lato $\{2,4\}$ fa scendere l'etichetta di 2 da 8 a 2: da quel momento 2 è più vicino all'albero passando da 4 che da 1.

Prim e Dijkstra (domanda da orale): stesso schema, etichette diverse.

	Dijkstra	**Prim**
grafo	orientato	non orientato
d_v è	il costo del **cammino** da r a v	il costo del **solo lato** che collega v all'albero
aggiorno se	$d_u + c_{uv} < d_v$	$c_{uv} < d_v$
nodo estratto	etichetta definitiva	etichetta $-M$: è nell'albero
minimizza	il costo di ogni cammino	la somma dei costi dei lati

Riepilogo

Trucchi di modellazione

voglio	**scrivo**
esattamente / almeno / al più uno	$\sum x = 1, \quad \geq 1, \quad \leq 1$
se A allora C	$x_A \leq x_C$
$C = A \wedge B$	$x_C \geq x_A + x_B - 1, x_C \leq x_A, x_C \leq x_B$
$C = A \vee B$	$x_C \geq x_A, x_C \geq x_B, x_C \leq x_A + x_B$
legare y binaria a x continua	$x \leq My$ (o $x \leq uy$ con la capacità)
valori discreti $d_1, \ldots, d_n$	$y = \sum d_i z_i, \sum z_i \leq 1$
semicontinua: 0 oppure $[l, u]$	$lz \leq y \leq uz$
funzione a tratti	una z^i e una w^i per tratto, $l_i z^i \leq w^i \leq u_i z^i, \sum z^i \leq 1$
$\min \max_i f_i(x)$	$\min v, v \geq f_i(x)$
$\max \min_i f_i(x)$	$\max v, v \leq f_i(x)$
$\min \lvert f(x) \rvert$	$\min v, v \geq f(x), v \geq -f(x)$
vale I oppure II	$I \leq b_I + My_I, II \leq b_{II} + My_{II}, y_I + y_{II} \leq 1$
prodotto di probabilità $\geq p$	$\sum (\log p_{ij}) x_{ij} \geq \log p$

Algoritmi sui grafi

algoritmo	**sceglie da Q**	**complessità**	**note**
visita	il primo (fila)	$O(m)$	dice se un nodo è raggiungibile
SPT.L, Bellman-Ford	il primo (fila)	$O(nm)$	costi negativi ammessi; nodo in coda n volte $\Rightarrow$ ciclo negativo
SPT.S, Dijkstra	etichetta minima	$O(n^2)$, heap $O(m \log n)$	solo costi ≥ 0; ogni nodo esce una volta
SPT.Acyclic	ordine topologico	$O(m)$	solo grafi aciclici; costi negativi ammessi
Kruskal	lati per costo crescente	$O(m \log n)$	stop a $n - 1$ lati
Prim	etichetta minima (costo del lato)		nodi dell'albero a $-M$